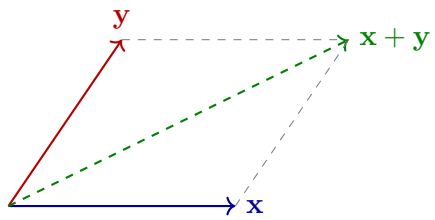
# Mathematics for Artificial Intelligence

A Beginner's Handbook




Course Handbook

April 29, 2026

# Preface

This handbook is for you if you are a curious undergraduate who wants to understand the mathematics that powers modern Artificial Intelligence, but who has not done a serious math course since high school. We will not pretend you remember every detail of calculus or linear algebra. We will start gently, build slowly, and connect every idea to a concrete AI scenario.

**Who this book is for.** A second-year college student who passed high-school algebra, trigonometry, and a single semester of calculus, then forgot most of it. You have used Python a little, watched a few AI videos online, and wondered what is really going on inside a neural network. This book is your map.

**Prerequisites.** Comfort with basic algebra (solving for $x$, manipulating fractions), exposure to the idea of a derivative, and willingness to sit with a confusing equation for ten minutes before giving up. That is all.

**How to read this book.** Each chapter follows the same rhythm: an opening hook from a real AI system, a short why-it-matters list, the core ideas with intuition before formula, derivations with no skipped steps, five worked examples, real-world applications, diagrams, common pitfalls, a summary, and ten homework problems. Read with a pencil. Do not skip the worked examples. The homework is where the learning lives.

**Recommended pace.** One chapter per week, six weeks for the whole book. If you spend two evenings on the reading and two on the homework, you will arrive at the end with a working understanding of the math behind transformers, gradient descent, Bayesian inference, and graph neural networks.

**What this book is not.** It is not a research monograph. It is not a substitute for a rigorous textbook on real analysis or measure-theoretic probability. It is a bridge: from where you are now to a place where deep learning papers stop looking like ancient runes.

Take your time. The mathematics in these pages is not difficult; it is unfamiliar. Familiarity is just practice in disguise.

# Contents


**Preface** **1**

**Notation Guide** **7**

**1 Linear Algebra** **8**
- 1.1 Opening Hook . . . 8
- 1.2 Why This Matters for AI . . . 8
- 1.3 Core Concepts . . . 9
  - 1.3.1 Vectors . . . 9
  - 1.3.2 Dot product . . . 9
  - 1.3.3 Matrices . . . 10
  - 1.3.4 Matrix multiplication . . . 10
  - 1.3.5 Transpose . . . 10
  - 1.3.6 Identity, inverse, and determinant . . . 10
  - 1.3.7 Rank . . . 11
  - 1.3.8 Eigenvalues and eigenvectors . . . 11
  - 1.3.9 Singular Value Decomposition . . . 11
  - 1.3.10 Vector spaces, projections, norms . . . 11
- 1.4 Step-by-Step Derivations . . . 11
  - 1.4.1 Derivation 1: The Cauchy–Schwarz inequality (special case) . . . 11
  - 1.4.2 Derivation 2: $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$ . . . 12
  - 1.4.3 Derivation 3: A $2 \times 2$ matrix is invertible iff $\det \neq 0$ . . . 12
  - 1.4.4 Derivation 4: The projection formula . . . 13
  - 1.4.5 Derivation 5: Why Symmetric Matrices Have Real Eigenvalues . . . 13
- 1.5 Worked Examples . . . 14
- 1.6 Real-World AI Applications . . . 15
- 1.7 Visual Explanation . . . 16
- 1.8 Common Pitfalls . . . 16
- 1.9 Chapter Summary . . . 17
- 1.10 Homework . . . 17

**2 Probability and Statistics** **18**
- 2.1 Opening Hook . . . 18
- 2.2 Why This Matters for AI . . . 18
- 2.3 Core Concepts . . . 18
  - 2.3.1 Sample spaces and events . . . 18
  - 2.3.2 Conditional probability and Bayes . . . 19
  - 2.3.3 Random variables . . . 19
  - 2.3.4 Expectation and variance . . . 19
  - 2.3.5 Common distributions . . . 19

2.3.6 Central Limit Theorem . . . 20
2.3.7 MLE and MAP . . . 20
2.3.8 Covariance and correlation . . . 20
2.4 Step-by-Step Derivations . . . 21
2.4.1 Derivation 1: Bayes' theorem from the definition of conditional probability 21
2.4.2 Derivation 2: Variance shortcut $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$ . . . 21
2.4.3 Derivation 3: MLE for a Gaussian mean . . . 21
2.4.4 Derivation 4: Linearity of expectation . . . 22
2.4.5 Derivation 5: Variance of a Sum of Independent Variables . . . 22
2.5 Worked Examples . . . 23
2.6 Real-World AI Applications . . . 24
2.7 Visual Explanation . . . 24
2.8 Common Pitfalls . . . 25
2.9 Chapter Summary . . . 25
2.10 Homework . . . 26

**3 Calculus** **27**
3.1 Opening Hook . . . 27
3.2 Why This Matters for AI . . . 27
3.3 Core Concepts . . . 27
3.3.1 Limits . . . 27
3.3.2 Derivatives . . . 27
3.3.3 Partial derivatives and the gradient . . . 28
3.3.4 Jacobian and Hessian . . . 28
3.3.5 Chain rule for many variables . . . 28
3.3.6 Taylor series . . . 29
3.3.7 Integration intuition . . . 29
3.4 Step-by-Step Derivations . . . 29
3.4.1 Derivation 1: Derivative of $\sigma(x) = 1/(1 + e^{-x})$ . . . 29
3.4.2 Derivation 2: Gradient of squared loss . . . 29
3.4.3 Derivation 3: Second-order Taylor expansion . . . 30
3.5 Worked Examples . . . 31
3.6 Real-World AI Applications . . . 32
3.7 Visual Explanation . . . 33
3.8 Common Pitfalls . . . 33
3.9 Chapter Summary . . . 33
3.10 Homework . . . 34

**4 Optimization** **35**
4.1 Opening Hook . . . 35
4.2 Why This Matters for AI . . . 35
4.3 Core Concepts . . . 35
4.3.1 Convexity . . . 35
4.3.2 Gradient descent . . . 36
4.3.3 Momentum . . . 36
4.3.4 Adam (intuition) . . . 36
4.3.5 Stochastic vs batch . . . 36
4.3.6 Local vs global minima . . . 36
4.3.7 Lagrange multipliers . . . 36
4.4 Step-by-Step Derivations . . . 37
4.4.1 Derivation 1: GD on a quadratic converges when $\eta < 2/L$ . . . 37
4.4.2 Derivation 2: First-order optimality of Lagrangian . . . 37

4.4.3 Derivation 3: Convexity of squared loss . . . 37
4.4.4 Derivation 4: Why Momentum Helps in a Narrow Valley . . . 37
4.4.5 Derivation 5: The Optimal Learning Rate for a Quadratic Bowl . . . 38
4.5 Worked Examples . . . 38
4.6 Real-World AI Applications . . . 40
4.7 Visual Explanation . . . 40
4.8 Common Pitfalls . . . 40
4.9 Chapter Summary . . . 41
4.10 Homework . . . 41

**5 Information Theory** **42**
5.1 Opening Hook . . . 42
5.2 Why This Matters for AI . . . 42
5.3 Core Concepts . . . 42
5.3.1 Entropy . . . 42
5.3.2 Cross-entropy . . . 43
5.3.3 KL divergence . . . 43
5.3.4 Mutual information . . . 43
5.3.5 Why log appears everywhere . . . 43
5.4 Step-by-Step Derivations . . . 43
5.4.1 Derivation 1: $D_{\mathrm{KL}}(p\|q) \geq 0$ . . . 43
5.4.2 Derivation 2: Cross-entropy = entropy + KL . . . 44
5.4.3 Derivation 3: Mutual information in entropy form . . . 44
5.4.4 Derivation 4: The Beautifully Simple Softmax + Cross-Entropy Gradient 44
5.4.5 Derivation 5: The Differential Entropy of a Gaussian . . . 45
5.5 Worked Examples . . . 46
5.6 Real-World AI Applications . . . 47
5.7 Visual Explanation . . . 47
5.8 Common Pitfalls . . . 47
5.9 Chapter Summary . . . 48
5.10 Homework . . . 48

**6 Discrete Mathematics** **50**
6.1 Opening Hook . . . 50
6.2 Why This Matters for AI . . . 50
6.3 Core Concepts . . . 50
6.3.1 Sets . . . 50
6.3.2 Logic . . . 51
6.3.3 Combinatorics . . . 51
6.3.4 Graphs . . . 51
6.3.5 Trees . . . 51
6.3.6 Big-O complexity . . . 51
6.4 Step-by-Step Derivations . . . 52
6.4.1 Derivation 1: $|A \cup B| = |A| + |B| - |A \cap B|$ . . . 52
6.4.2 Derivation 2: Number of subsets is $2^n$ . . . 52
6.4.3 Derivation 3: Tree on $n$ vertices has $n-1$ edges . . . 52
6.4.4 Derivation 4: The Handshake Lemma . . . 52
6.4.5 Derivation 5: Why Binary Search is $O(\log n)$ . . . 53
6.5 Worked Examples . . . 53
6.6 Real-World AI Applications . . . 54
6.7 Visual Explanation . . . 54
6.8 Common Pitfalls . . . 54

6.9 Chapter Summary . . . . . 55
6.10 Homework . . . . . 55

**A Math Refresher** **57**
A.1 High-school algebra . . . . . 57
A.2 Trigonometry essentials . . . . . 58

**B Greek Alphabet Reference** **59**

**C Further Reading** **60**

# List of Figures


1.1 Vector addition by the parallelogram rule. . . . . 16
1.2 A matrix transforms a unit square into a parallelogram. The signed area of the parallelogram is the determinant. . . . . 16

2.1 Two Gaussian densities. Larger variance flattens and widens the curve. . . . . 24
2.2 The Binomial$(5, 0.5)$ probability mass function. . . . . 25

3.1 The tangent line at $x = 1$ has slope $f'(1) = 2$. . . . . 33
3.2 Slopes of $y = 0.4x^2$ at sample points; the derivative is the local slope. . . . . 33

4.1 Gradient descent on $f(w) = (w-3)^2$ with $\eta = 0.1$ converges towards the minimum at $w = 3$. . . . . 40
4.2 Convex (blue) versus non-convex (red): the convex bowl has a unique global minimum; the wavy curve has many local minima. . . . . 40

5.1 Binary entropy peaks at $p = 0.5$ (the fair coin) and is zero at $p \in \{0, 1\}$. . . . . 47
5.2 Two Gaussians with the same variance; their KL divergence is $\frac{1}{2} \cdot 1 = 0.5$ nats. . 48

6.1 An undirected graph with five vertices and five edges. . . . . 55
6.2 A rooted tree: 6 vertices, 5 edges, exactly one path between any pair. . . . . 55

# Notation Guide

The table below lists conventions used throughout the book. Glance at it now, and refer back when a symbol confuses you.

| Symbol | Meaning |
|---|---|
| $x, y, \alpha, \theta$ | Scalars (italic, lowercase) |
| $\mathbf{x}, \mathbf{y}, \mathbf{w}$ | Vectors (bold, lowercase) |
| $\mathbf{A}, \mathbf{B}, \mathbf{X}$ | Matrices (bold, uppercase) |
| $\mathbf{A}^\top$ | Transpose of matrix $\mathbf{A}$ |
| $\mathbf{A}^{-1}$ | Inverse of matrix $\mathbf{A}$ |
| $\det(\mathbf{A})$ or $\lvert\mathbf{A}\rvert$ | Determinant of $\mathbf{A}$ |
| $\lVert\mathbf{x}\rVert$ | Euclidean norm of vector $\mathbf{x}$ |
| $\langle\mathbf{x}, \mathbf{y}\rangle$ or $\mathbf{x}^\top\mathbf{y}$ | Dot product |
| $\mathbb{R}^n$ | The set of $n$-dimensional real vectors |
| $\mathbb{E}[X]$ | Expected value of random variable $X$ |
| $\mathrm{Var}(X)$ | Variance of $X$ |
| $P(A)$ | Probability of event $A$ |
| $P(A \mid B)$ | Probability of $A$ given $B$ |
| $\sim$ | "Is distributed as" (e.g. $X \sim \mathcal{N}(0, 1)$) |
| $\mathcal{N}(\mu, \sigma^2)$ | Gaussian/normal distribution |
| $\nabla f$ | Gradient of $f$ |
| $\frac{\partial f}{\partial x}$ | Partial derivative of $f$ with respect to $x$ |
| $D_{\mathrm{KL}}(P\lVert Q)$ | Kullback–Leibler divergence |
| $H(X)$ | Entropy of $X$ |
| $\log$ | Natural logarithm (base $e$) unless noted |
| $\mathbb{1}[\cdot]$ | Indicator function (1 if true, 0 if false) |
| $:=$ | "Defined as" |

# Chapter 1

# Linear Algebra

## 1.1 Opening Hook

When ChatGPT reads your message, every word becomes a vector of 12,288 numbers. The word "cat" might be $[0.21, -0.04, \ldots, 1.13]$. The word "feline" lives nearby in this 12,288-dimensional space. The word "democracy" lives far away. To find which words are similar, the model computes *dot products*. To compress a vector down to a smaller representation, it multiplies by a *matrix*. To rotate, scale, or project meaning, it again multiplies by matrices. Linear algebra is not a tool inside transformers; it *is* the transformer. Strip away the linear algebra and what remains is a non-linear squashing function and a softmax. Everything else is vectors and matrices, all the way down.

## 1.2 Why This Matters for AI

Linear algebra is the language in which neural networks are written. Here is why the chapter you are about to read will repay every minute you spend on it.

**Words become vectors.** Every modern language model represents tokens as high-dimensional vectors. Adding, scaling, and projecting these vectors are operations you will do thousands of times when reasoning about embeddings.

**Layers are matrix multiplications.** A fully connected layer of a neural network is the operation $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$. Stack many such layers and you get a deep network. To debug, train, or speed up such a network, you must understand the underlying matrix algebra.

**Eigenvectors reveal structure.** PageRank ranks web pages using the principal eigenvector of a matrix. Principal Component Analysis compresses data using eigenvectors of the covariance matrix. Graph neural networks use spectral properties of the Laplacian.

**SVD compresses everything.** The Singular Value Decomposition underpins dimensionality reduction, latent semantic analysis, recommender systems, and the compression of large language models.

**Norms measure progress.** When training a network, you measure error as a norm of a difference. When regularizing, you penalize a norm of the weights. Norms are how loss functions speak.

## 1.3 Core Concepts

### 1.3.1 Vectors

**Intuition**

A vector is an arrow in space. It has a length and a direction. We write it as a list of numbers, but the list is just a way to record the arrow on paper.

**Definition 1.1** (Vector)**.** A vector $\mathbf{x} \in \mathbb{R}^n$ is an ordered list of $n$ real numbers, written as a column:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

The plain-English version: a vector of length $n$ is just $n$ numbers in a fixed order. The geometry comes from how we add them and stretch them.

**Addition.** You add vectors component-by-component:

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}.$$

Geometrically: lay the second arrow at the tip of the first.

**Scalar multiplication.** You stretch a vector by multiplying every entry:

$$2 \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}.$$

Geometrically: same direction, twice the length.

### 1.3.2 Dot product

**Definition 1.2** (Dot product)**.** For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$,

$$\mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^{n} x_i y_i.$$

In words: multiply matching entries and add them up. The result is one number.

There is a second formula for the same quantity, and it is the geometric one:

$$\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \, \|\mathbf{y}\| \cos\theta, \tag{1.1}$$

where $\theta$ is the angle between the two arrows. The dot product equals the product of lengths times the cosine of the angle between them. When the vectors point the same way, $\cos\theta = 1$ and the dot product is large. When they are perpendicular, $\cos\theta = 0$ and the dot product is zero.

**AI Connection**

The cosine similarity between two word embeddings $\mathbf{u}$ and $\mathbf{v}$ is exactly

$$\cos\theta = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}.$$

This is how a search engine decides that "car" and "automobile" are close.

### 1.3.3 Matrices

**Definition 1.3** (Matrix)**.** A matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ is a rectangular array of real numbers with $m$ rows and $n$ columns:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

A matrix is a grid of numbers, but it is more usefully thought of as a function. A matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ takes a vector $\mathbf{x} \in \mathbb{R}^n$ and produces a vector $\mathbf{Ax} \in \mathbb{R}^m$.

### 1.3.4 Matrix multiplication

For $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times p}$, the product $\mathbf{C} = \mathbf{AB}$ is the $m \times p$ matrix with entries

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

In plain English: entry $(i, j)$ of the product is the dot product of row $i$ of $\mathbf{A}$ with column $j$ of $\mathbf{B}$.

**Important**

For matrix multiplication to be defined, the number of columns of the left matrix must match the number of rows of the right matrix. The product of $m \times n$ and $n \times p$ is $m \times p$. The inner dimensions must agree; they vanish in the product.

### 1.3.5 Transpose

The transpose $\mathbf{A}^\top$ flips the matrix across its diagonal. Row $i$ of $\mathbf{A}$ becomes column $i$ of $\mathbf{A}^\top$. So $(\mathbf{A}^\top)_{ij} = a_{ji}$. Two facts you will use forever:

$$(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top, \qquad (\mathbf{A}^\top)^\top = \mathbf{A}.$$

### 1.3.6 Identity, inverse, and determinant

The identity matrix $\mathbf{I}_n$ is the $n \times n$ matrix with ones on the diagonal and zeros elsewhere; it is the "do nothing" matrix: $\mathbf{Ix} = \mathbf{x}$.

The inverse $\mathbf{A}^{-1}$, when it exists, satisfies $\mathbf{AA}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$. Not every matrix has an inverse. A square matrix is invertible if and only if its determinant is nonzero.

The determinant $\det(\mathbf{A})$ is one number that summarizes a matrix. For a $2 \times 2$ matrix,

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc.$$

Geometrically, $|\det(\mathbf{A})|$ is the volume by which $\mathbf{A}$ scales space. If $\det(\mathbf{A}) = 0$, the matrix squashes space onto a lower-dimensional subspace and information is lost; the matrix cannot be inverted.

### 1.3.7 Rank

The rank of a matrix is the number of linearly independent columns it has. Equivalently, it is the dimension of the subspace spanned by the columns. Full rank means no column is a redundant copy of the others. Low rank means redundancy, which is what compression exploits.

### 1.3.8 Eigenvalues and eigenvectors

**Definition 1.4** (Eigenvalue, eigenvector)**.** For a square matrix $\mathbf{A}$, a nonzero vector $\mathbf{v}$ is an eigenvector with eigenvalue $\lambda$ if

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}.$$

In plain English: $\mathbf{v}$ is a special direction along which $\mathbf{A}$ acts purely as a stretch. The matrix does not rotate $\mathbf{v}$; it just scales it by $\lambda$.

To find them: solve $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$ for $\lambda$, then for each $\lambda$ solve $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$.

### 1.3.9 Singular Value Decomposition

Every real matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ can be written

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top,$$

where $\mathbf{U}$ is $m \times m$ orthogonal, $\mathbf{V}$ is $n \times n$ orthogonal, and $\mathbf{\Sigma}$ is $m \times n$ with non-negative numbers on its diagonal called the singular values, sorted from largest to smallest. The SVD is the most useful factorization in applied math: it tells you how a matrix stretches space, in which directions, and by how much.

### 1.3.10 Vector spaces, projections, norms

A *vector space* is a set of vectors closed under addition and scaling. The Euclidean norm of a vector is

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}.$$

Other norms exist; the $L_1$ norm is $\sum |x_i|$, the $L_\infty$ norm is $\max_i |x_i|$.

The projection of $\mathbf{x}$ onto a unit vector $\mathbf{u}$ is

$$\mathrm{proj}_{\mathbf{u}}(\mathbf{x}) = (\mathbf{x} \cdot \mathbf{u})\mathbf{u}.$$

This says: take the component of $\mathbf{x}$ along $\mathbf{u}$, and re-attach it to $\mathbf{u}$.

## 1.4 Step-by-Step Derivations

### 1.4.1 Derivation 1: The Cauchy–Schwarz inequality (special case)

We will show that for any two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$,

$$|\mathbf{x} \cdot \mathbf{y}| \leq \|\mathbf{x}\|\|\mathbf{y}\|.$$

Start from the obvious fact that the squared norm of any real vector is non-negative:

$$
\begin{aligned}
0 &\le \|\mathbf{x} - t\mathbf{y}\|^2 && \text{(squared norms are non-negative)} && (1.2)\\
&= (\mathbf{x} - t\mathbf{y}) \cdot (\mathbf{x} - t\mathbf{y}) && \text{(definition of squared norm)} && (1.3)\\
&= \mathbf{x} \cdot \mathbf{x} - 2t(\mathbf{x} \cdot \mathbf{y}) + t^2(\mathbf{y} \cdot \mathbf{y}) && \text{(distribute the dot product)} && (1.4)\\
&= \|\mathbf{x}\|^2 - 2t(\mathbf{x} \cdot \mathbf{y}) + t^2\|\mathbf{y}\|^2. && \text{(rewrite squared norms)} && (1.5)
\end{aligned}
$$

This is a quadratic in $t$ that is non-negative for every real $t$. Its discriminant must be non-positive:

$$
\begin{aligned}
(2\mathbf{x} \cdot \mathbf{y})^2 - 4\|\mathbf{x}\|^2\|\mathbf{y}\|^2 &\le 0 && \text{(discriminant condition)} && (1.6)\\
(\mathbf{x} \cdot \mathbf{y})^2 &\le \|\mathbf{x}\|^2\|\mathbf{y}\|^2 && \text{(divide by 4)} && (1.7)\\
|\mathbf{x} \cdot \mathbf{y}| &\le \|\mathbf{x}\|\|\mathbf{y}\|. && \text{(take square roots)} && (1.8)
\end{aligned}
$$

Combined with equation (1.1), this is exactly the statement that $|\cos\theta| \le 1$.

### 1.4.2 Derivation 2: $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$

Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times p}$. The $(i, j)$ entry of $\mathbf{AB}$ is

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj}.$$

Now compute the $(i, j)$ entry of $(\mathbf{AB})^\top$:

$$
\begin{aligned}
((\mathbf{AB})^\top)_{ij} &= (\mathbf{AB})_{ji} && \text{(definition of transpose)} && (1.9)\\
&= \sum_{k=1}^{n} a_{jk}b_{ki} && \text{(definition of matrix multiplication)} && (1.10)\\
&= \sum_{k=1}^{n} b_{ki}a_{jk} && \text{(reorder real numbers)} && (1.11)\\
&= \sum_{k=1}^{n} (\mathbf{B}^\top)_{ik}(\mathbf{A}^\top)_{kj} && \text{(rewrite each factor as a transpose entry)} && (1.12)\\
&= (\mathbf{B}^\top\mathbf{A}^\top)_{ij}. && \text{(definition of matrix multiplication)} && (1.13)
\end{aligned}
$$

Two matrices that match entry-by-entry are equal, so $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$.

### 1.4.3 Derivation 3: A $2 \times 2$ matrix is invertible iff $\det \neq 0$

Take

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad \det(\mathbf{A}) = ad - bc.$$

We claim that when $ad - bc \neq 0$,

$$\mathbf{A}^{-1} = \frac{1}{ad - bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Verify by multiplying:

$$\begin{aligned}
\mathbf{A}\mathbf{A}^{-1} &= \frac{1}{ad-bc}\begin{bmatrix} a & b \\ c & d \end{bmatrix}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix} && \text{(plug in)} && (1.14)\\
&= \frac{1}{ad-bc}\begin{bmatrix} ad-bc & -ab+ab \\ cd-cd & -bc+ad \end{bmatrix} && \text{(carry out the multiplication)} && (1.15)\\
&= \frac{1}{ad-bc}\begin{bmatrix} ad-bc & 0 \\ 0 & ad-bc \end{bmatrix} && \text{(simplify)} && (1.16)\\
&= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \mathbf{I}. && \text{(divide by the scalar)} && (1.17)
\end{aligned}$$

Conversely, if $ad - bc = 0$, the rows are linearly dependent, the rank is at most one, and no inverse exists.

### 1.4.4 Derivation 4: The projection formula

We want to project a vector $\mathbf{x}$ onto the line spanned by a non-zero vector $\mathbf{u}$. Geometrically, the projection $\mathbf{p}$ is the point on the line closest to $\mathbf{x}$. The vector from $\mathbf{p}$ to $\mathbf{x}$ is perpendicular to the line, so it must be perpendicular to $\mathbf{u}$.

Write $\mathbf{p} = c\mathbf{u}$ for some scalar $c$ to be determined. Perpendicularity says

$$\begin{aligned}
(\mathbf{x} - c\mathbf{u}) \cdot \mathbf{u} &= 0 && \text{(perpendicularity)} && (1.18)\\
\mathbf{x} \cdot \mathbf{u} - c(\mathbf{u} \cdot \mathbf{u}) &= 0 && \text{(distribute the dot product)} && (1.19)\\
c &= \frac{\mathbf{x} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}}. && \text{(solve for } c\text{)} && (1.20)
\end{aligned}$$

Therefore

$$\operatorname{proj}_{\mathbf{u}}(\mathbf{x}) = \frac{\mathbf{x} \cdot \mathbf{u}}{\|\mathbf{u}\|^2}\,\mathbf{u}.$$

When $\mathbf{u}$ is a unit vector ($\|\mathbf{u}\| = 1$), this simplifies to $(\mathbf{x} \cdot \mathbf{u})\mathbf{u}$, which appears in self-attention scoring.

### 1.4.5 Derivation 5: Why Symmetric Matrices Have Real Eigenvalues

Symmetric matrices ($\mathbf{A}^\top = \mathbf{A}$) are everywhere in machine learning: covariance matrices, Gram matrices $\mathbf{X}^\top\mathbf{X}$, Hessians of twice-differentiable functions. They have a remarkable property: all their eigenvalues are real numbers, never complex. This makes algorithms like PCA stable and interpretable. Here is why.

Suppose $\mathbf{A}$ is a real symmetric matrix and $\lambda$ is an eigenvalue with eigenvector $\mathbf{v}$, possibly complex. So $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. Take the conjugate transpose (denoted $^*$, which means transpose then take complex conjugate of each entry) of both sides. Since $\mathbf{A}$ has real entries, $\mathbf{A}^* = \mathbf{A}^\top = \mathbf{A}$:

$$\begin{aligned}
\mathbf{A}\mathbf{v} &= \lambda\mathbf{v} && \text{eigenvalue equation,} && (1.21)\\
\mathbf{v}^*\mathbf{A}\mathbf{v} &= \lambda\mathbf{v}^*\mathbf{v} && \text{left-multiply by } \mathbf{v}^*. && (\star)
\end{aligned}$$

Now consider $(\mathbf{v}^*\mathbf{A}\mathbf{v})^*$. The conjugate transpose of a $1 \times 1$ scalar is its complex conjugate, so this equals $\overline{\mathbf{v}^*\mathbf{A}\mathbf{v}}$. But also $(\mathbf{v}^*\mathbf{A}\mathbf{v})^* = \mathbf{v}^*\mathbf{A}^*\mathbf{v} = \mathbf{v}^*\mathbf{A}\mathbf{v}$ because $\mathbf{A}$ is real symmetric. So $\mathbf{v}^*\mathbf{A}\mathbf{v}$ equals its own complex conjugate, which means it is real.

Now look at $(\star)$. The left side is real (just shown). The right side is $\lambda \cdot \mathbf{v}^*\mathbf{v}$, where $\mathbf{v}^*\mathbf{v} = \sum_i |v_i|^2$ is a positive real number. A real number divided by a positive real number is real, so $\lambda$ itself must be real. $\square$

The same argument shows eigenvectors corresponding to distinct eigenvalues are orthogonal. The full statement — a real symmetric matrix is diagonalizable by an orthogonal matrix with real eigenvalues — is the *spectral theorem*, and it's the mathematical foundation of PCA, spectral clustering, and quadratic-form analysis.

## 1.5 Worked Examples

**Example 1.1: Computing a dot product**

Let $\mathbf{x} = (1, 2, 3)$ and $\mathbf{y} = (4, -1, 2)$. Compute $\mathbf{x} \cdot \mathbf{y}$ and the angle between them.
**Solution.** Multiply matching entries and add:

$$\begin{aligned}\mathbf{x} \cdot \mathbf{y} &= (1)(4) + (2)(-1) + (3)(2) \\ &= 4 - 2 + 6 = 8.\end{aligned}$$

Now compute the norms:

$$\|\mathbf{x}\| = \sqrt{1 + 4 + 9} = \sqrt{14}, \quad \|\mathbf{y}\| = \sqrt{16 + 1 + 4} = \sqrt{21}.$$

Therefore $\cos\theta = \frac{8}{\sqrt{14}\sqrt{21}} = \frac{8}{\sqrt{294}} \approx 0.467$, so $\theta \approx 62.2°$.

**Example 1.2: Matrix-vector product**

Compute $\mathbf{Ax}$ for

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ -1 & 3 \\ 0 & 4 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}.$$

**Solution.** Each row of $\mathbf{A}$ dotted with $\mathbf{x}$ gives one entry of the answer:

$$\begin{aligned}\mathbf{Ax} &= \begin{bmatrix} (2)(5) + (1)(2) \\ (-1)(5) + (3)(2) \\ (0)(5) + (4)(2) \end{bmatrix} \\ &= \begin{bmatrix} 10 + 2 \\ -5 + 6 \\ 0 + 8 \end{bmatrix} = \begin{bmatrix} 12 \\ 1 \\ 8 \end{bmatrix}.\end{aligned}$$

**Example 1.3: Matrix-matrix product**

Multiply

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

**Solution.** Entry $(i, j)$ of $\mathbf{AB}$ is row $i$ of $\mathbf{A}$ dotted with column $j$ of $\mathbf{B}$:

$$\begin{aligned}(\mathbf{AB})_{11} &= (1)(5) + (2)(7) = 5 + 14 = 19 \\ (\mathbf{AB})_{12} &= (1)(6) + (2)(8) = 6 + 16 = 22 \\ (\mathbf{AB})_{21} &= (3)(5) + (4)(7) = 15 + 28 = 43 \\ (\mathbf{AB})_{22} &= (3)(6) + (4)(8) = 18 + 32 = 50.\end{aligned}$$

Therefore $\mathbf{AB} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$.

**Example 1.4: Inverse of a $2 \times 2$ matrix**

Find the inverse of $\mathbf{A} = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}$.

**Solution.** The determinant is $\det(\mathbf{A}) = (4)(6) - (7)(2) = 24 - 14 = 10$. Apply the formula:

$$\mathbf{A}^{-1} = \frac{1}{10} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} = \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix}.$$

Quick check:

$$\mathbf{A}\mathbf{A}^{-1} = \begin{bmatrix} (4)(0.6) + (7)(-0.2) & (4)(-0.7) + (7)(0.4) \\ (2)(0.6) + (6)(-0.2) & (2)(-0.7) + (6)(0.4) \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

**Example 1.5: Eigenvalues of a $2 \times 2$ matrix**

Find the eigenvalues and eigenvectors of $\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}$.

**Solution.** Solve $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$:

$$\det \begin{bmatrix} 3 - \lambda & 1 \\ 0 & 2 - \lambda \end{bmatrix} = (3 - \lambda)(2 - \lambda) - 0 = 0.$$

So $\lambda_1 = 3$ and $\lambda_2 = 2$.

For $\lambda_1 = 3$: solve $\begin{bmatrix} 0 & 1 \\ 0 & -1 \end{bmatrix} \mathbf{v} = \mathbf{0}$, giving $v_2 = 0$ and $v_1$ free. Take $\mathbf{v}_1 = (1, 0)$.

For $\lambda_2 = 2$: solve $\begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \mathbf{v} = \mathbf{0}$, giving $v_1 + v_2 = 0$. Take $\mathbf{v}_2 = (1, -1)$.

Verify: $\mathbf{A}\mathbf{v}_1 = (3, 0)^\top = 3\mathbf{v}_1$ and $\mathbf{A}\mathbf{v}_2 = (2, -2)^\top = 2\mathbf{v}_2$. ✓

## 1.6 Real-World AI Applications

**word2vec.** Each word in a vocabulary is assigned a dense vector. The model is trained so that words appearing in similar contexts get nearby vectors. The famous arithmetic "king − man + woman ≈ queen" is just vector addition. Cosine similarity (a normalized dot product) is the workhorse for retrieving similar words.

**Transformer attention.** The attention mechanism computes scores $\mathbf{Q}\mathbf{K}^\top/\sqrt{d}$, applies softmax row-wise, then multiplies by $\mathbf{V}$. The whole operation is three matrix multiplications and a rescaling. Understanding why $\mathbf{Q}\mathbf{K}^\top$ produces a similarity matrix requires only the dot product viewed as "how aligned are these two vectors?".

**PageRank.** Google's original ranking algorithm computes the dominant eigenvector of a stochastic matrix derived from the web's hyperlink structure. The eigenvector entries are page importance scores. Power iteration, a beginner-friendly algorithm, finds this eigenvector by repeatedly multiplying any starting vector by the matrix.

**Principal Component Analysis (PCA).** Given a data matrix $\mathbf{X}$, PCA finds the directions of maximum variance by computing the eigenvectors of the covariance matrix $\mathbf{X}^\top\mathbf{X}/(n-1)$. Equivalently, it uses the SVD of $\mathbf{X}$. PCA underlies eigenfaces, latent semantic analysis, and many compression schemes.

## 1.7 Visual Explanation

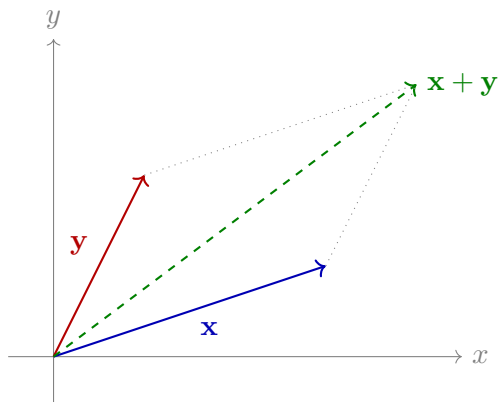



Figure 1.1: Vector addition by the parallelogram rule.

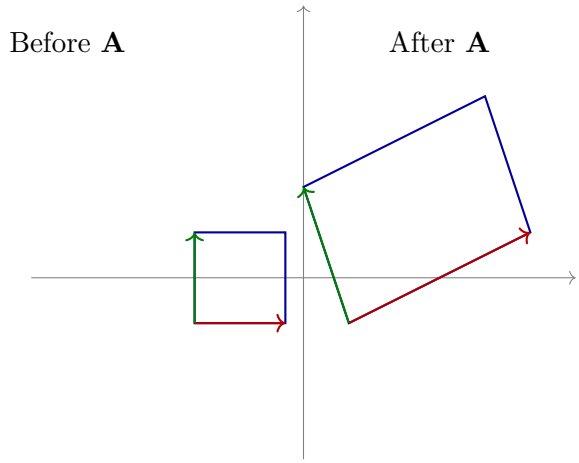



Figure 1.2: A matrix transforms a unit square into a parallelogram. The signed area of the parallelogram is the determinant.

## 1.8 Common Pitfalls

**Pitfall 1: Confusing rows and columns.** A matrix in $\mathbb{R}^{m\times n}$ has $m$ rows and $n$ columns. When multiplying, the inner dimensions must match. Many bugs in deep learning come from a transposed weight matrix.

**Pitfall 2: Treating matrix multiplication as commutative.** In general $\mathbf{AB} \neq \mathbf{BA}$. Test on small examples to convince yourself.

**Pitfall 3: Forgetting that determinant zero means non-invertible.** If a system of equations has determinant zero, you cannot uniquely solve it. The system either has no solutions or infinitely many.

**Pitfall 4: Computing the dot product as a matrix product.** The dot product of two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ is $\mathbf{x}^\top\mathbf{y}$, a scalar. The outer product $\mathbf{x}\mathbf{y}^\top$ is an $n \times n$ matrix. They are not the same; the order matters.

**Pitfall 5: Misusing eigenvectors of non-square matrices.** Eigenvalues are only defined for square matrices. For rectangular matrices use singular values.

**Pitfall 6: Ignoring numerical scaling.** Norms can blow up or vanish in high dimensions. Always normalize embeddings before computing cosine similarity.

## 1.9 Chapter Summary

- A vector is an ordered tuple of real numbers; a matrix is a rectangular grid.
- Matrix multiplication: $(\mathbf{AB})_{ij}$ is row $i$ of $\mathbf{A}$ dotted with column $j$ of $\mathbf{B}$. Inner dimensions must match.
- Transpose flips across the diagonal: $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$.
- Inverse exists when det $\neq 0$; for $2 \times 2$, $\mathbf{A}^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$.
- Eigenvalue equation: $\mathbf{Av} = \lambda\mathbf{v}$.
- SVD: $\mathbf{A} = \mathbf{U\Sigma V}^\top$.
- Cosine similarity:

$$\boxed{\cos\theta = \frac{\mathbf{x}\cdot\mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|}.}$$

## 1.10 Homework

**Problem 1.** **[Easy]** Compute $\mathbf{x}\cdot\mathbf{y}$ for $\mathbf{x} = (2, -1, 3)$ and $\mathbf{y} = (0, 4, -2)$.

**Problem 2.** **[Easy]** Find the norm of $\mathbf{x} = (3, 4, 12)$.

**Problem 3.** **[Easy]** Compute $\mathbf{A}^\top$ for $\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$.

**Problem 4.** **[Medium]** Find all eigenvalues of $\mathbf{A} = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$.

**Problem 5.** **[Medium]** Verify that $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$ on $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$.

**Problem 6.** **[Medium, AI]** Two word embeddings are $\mathbf{u} = (0.5, -0.2, 0.8, 0.1)$ and $\mathbf{v} = (0.4, -0.1, 0.7, 0.0)$. Compute the cosine similarity.

**Problem 7.** **[Medium, AI]** A neural-network layer has weight matrix $\mathbf{W} \in \mathbb{R}^{3\times 4}$ and bias $\mathbf{b} \in \mathbb{R}^3$. Given $\mathbf{W} = \begin{bmatrix} 1 & 0 & -1 & 2 \\ 0 & 1 & 1 & 0 \\ 2 & -1 & 0 & 1 \end{bmatrix}$, $\mathbf{b} = (1, 0, -1)$, and input $\mathbf{x} = (1, 1, 1, 1)$, compute $\mathbf{y} = \mathbf{Wx} + \mathbf{b}$.

**Problem 8.** **[Hard]** Show that if $\mathbf{A}$ is invertible, then $(\mathbf{A}^\top)^{-1} = (\mathbf{A}^{-1})^\top$.

**Problem 9.** **[Hard]** Prove that the eigenvalues of a symmetric matrix are real. (Hint: use complex conjugates.)

**Problem 10.** **[Hard, AI]** A user-item rating matrix $\mathbf{R}$ has rank 2. Argue why approximating $\mathbf{R}$ as $\mathbf{UV}^\top$ with $\mathbf{U} \in \mathbb{R}^{m\times 2}$, $\mathbf{V} \in \mathbb{R}^{n\times 2}$ is exactly equivalent to truncated SVD with $k = 2$.

# Chapter 2

# Probability and Statistics

## 2.1 Opening Hook

When a self-driving car sees a blurry shape on the road, it does not know with certainty whether the shape is a pedestrian, a plastic bag, or a shadow. It assigns a probability to each possibility, weighs the cost of each mistake, and acts. When a spam filter labels an email, it computes $P(\text{spam} \mid \text{words})$ using Bayes' theorem. When ChatGPT picks the next token, it samples from a probability distribution over 50,000 tokens. Probability is how an AI makes decisions in a world it cannot fully see.

## 2.2 Why This Matters for AI

**Models output probabilities.** Classification networks end with a softmax that produces a probability distribution. To train them, you compare predicted to true distributions using losses derived from probability theory.

**Bayes' theorem is everywhere.** Spam filters, medical diagnosis nets, and Bayesian neural networks all update beliefs using the same rule: posterior is proportional to likelihood times prior.

**Sampling drives generative AI.** Diffusion models, language models, and variational autoencoders all draw samples from learned distributions. You cannot understand them without understanding distributions.

**Statistics validates models.** Confidence intervals, hypothesis tests, and bootstrapping tell you whether a model improvement is real or just noise.

**The Central Limit Theorem hides inside training.** Stochastic gradient descent works in part because gradient estimates are approximately Gaussian-distributed around the true gradient.

## 2.3 Core Concepts

### 2.3.1 Sample spaces and events

**Definition 2.1** (Sample space, event)**.** A *sample space* $\Omega$ is the set of all possible outcomes of an experiment. An *event* is a subset of $\Omega$.

For a single die toss, $\Omega = \{1, 2, 3, 4, 5, 6\}$. The event "even" is $\{2, 4, 6\}$.

A *probability* assigns each event a number $P(A) \in [0,1]$, with $P(\Omega) = 1$ and the rule that probabilities of disjoint events add: $P(A \cup B) = P(A) + P(B)$ when $A$ and $B$ have no outcomes in common.

### 2.3.2 Conditional probability and Bayes

**Definition 2.2** (Conditional probability)**.** For events $A$ and $B$ with $P(B) > 0$,

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

In words: among the worlds where $B$ happens, what fraction also have $A$?

**Theorem 2.1** (Bayes' theorem)**.** *For events $A$ and $B$ with $P(A), P(B) > 0$,*

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}.$$

This is the most important formula in this book. It tells you how to update a prior belief $P(A)$ into a posterior $P(A \mid B)$ after seeing evidence $B$.

**AI Connection**

A spam filter computes $P(\text{spam} \mid \text{words})$. Direct measurement is hopeless because the space of possible word sequences is astronomical. Bayes inverts the question: estimate $P(\text{words} \mid \text{spam})$ from a corpus, multiply by $P(\text{spam})$, and divide by $P(\text{words})$. This is the naive Bayes classifier.

### 2.3.3 Random variables

A *random variable* is a function from outcomes to numbers. "The number of heads in 10 coin flips" is a random variable: each outcome (a sequence of H's and T's) gets mapped to a count.

A discrete random variable has a probability mass function (pmf): $p(x) = P(X = x)$. A continuous one has a probability density function (pdf): $f(x)$ such that $P(a \le X \le b) = \int_a^b f(x)\, dx$.

### 2.3.4 Expectation and variance

**Definition 2.3** (Expectation)**.** $\mathbb{E}[X] = \sum_x x\, p(x)$ for discrete $X$, or $\int x\, f(x)\, dx$ for continuous $X$.

The expectation is the long-run average. If you repeated the experiment many times, the average of the outcomes would tend to $\mathbb{E}[X]$.

**Definition 2.4** (Variance)**.** $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$.

The variance measures how much $X$ wiggles around its mean. The standard deviation $\sigma = \sqrt{\mathrm{Var}(X)}$ has the same units as $X$.

### 2.3.5 Common distributions

**Bernoulli**$(p)$**.** A coin flip with bias $p$. $P(X = 1) = p$, $P(X = 0) = 1 - p$. Mean $p$, variance $p(1-p)$.

**Binomial**$(n, p)$**.** Number of successes in $n$ independent Bernoulli trials. $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$. Mean $np$, variance $np(1 - p)$.

**Gaussian/Normal**$(\mu, \sigma^2)$**.**

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Mean $\mu$, variance $\sigma^2$. The Gaussian is the central distribution of statistics.

**Poisson**$(\lambda)$**.** Counts of rare events: $P(X = k) = e^{-\lambda}\lambda^k/k!$. Mean and variance both $\lambda$.

### 2.3.6 Central Limit Theorem

**Theorem 2.2** (Central Limit Theorem)**.** *Let $X_1, \ldots, X_n$ be independent and identically distributed with mean $\mu$ and variance $\sigma^2 < \infty$. Then*

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1) \quad \textit{as } n \to \infty,$$

*where $\bar{X}_n = \frac{1}{n}\sum_i X_i$.*

In plain English: averages of many independent random variables are approximately Gaussian, no matter what distribution the individual variables have.

### 2.3.7 MLE and MAP

Given data $\mathcal{D} = \{x_1, \ldots, x_n\}$ assumed to come from a distribution $p(x; \theta)$ with unknown parameter $\theta$:

**Maximum likelihood estimation (MLE):**

$$\hat{\theta}_{\text{MLE}} = \arg\max_\theta \prod_{i=1}^n p(x_i; \theta) = \arg\max_\theta \sum_{i=1}^n \log p(x_i; \theta).$$

Pick the parameter that makes the observed data most probable.

**Maximum a posteriori (MAP):**

$$\hat{\theta}_{\text{MAP}} = \arg\max_\theta p(\theta \mid \mathcal{D}) = \arg\max_\theta \big[\log p(\mathcal{D} \mid \theta) + \log p(\theta)\big].$$

MLE plus a prior log-probability acting as a regularizer.

### 2.3.8 Covariance and correlation

For two random variables $X, Y$:

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])], \quad \rho_{XY} = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}}.$$

Correlation $\rho \in [-1, 1]$ measures how linearly related $X$ and $Y$ are.

## 2.4 Step-by-Step Derivations

### 2.4.1 Derivation 1: Bayes' theorem from the definition of conditional probability

By definition,

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \qquad \text{(definition)} \tag{2.1}$$

$$P(B \mid A) = \frac{P(A \cap B)}{P(A)} \qquad \text{(definition)} \tag{2.2}$$

Solve the second equation for $P(A \cap B)$:

$$P(A \cap B) = P(B \mid A)\, P(A). \qquad \text{(rearrange)} \tag{2.3}$$

Substitute into the first:

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}. \qquad \text{(substitute)} \tag{2.4}$$

That is Bayes' theorem.

### 2.4.2 Derivation 2: Variance shortcut $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$

Let $\mu = \mathbb{E}[X]$. Expand:

$$\begin{aligned}
\mathrm{Var}(X) &= \mathbb{E}[(X-\mu)^2] && \text{(definition)} && (2.5)\\
&= \mathbb{E}[X^2 - 2\mu X + \mu^2] && \text{(expand the square)} && (2.6)\\
&= \mathbb{E}[X^2] - 2\mu\,\mathbb{E}[X] + \mu^2 && \text{(linearity of expectation; } \mu \text{ is a constant)} && (2.7)\\
&= \mathbb{E}[X^2] - 2\mu \cdot \mu + \mu^2 && \text{(by definition of } \mu\text{)} && (2.8)\\
&= \mathbb{E}[X^2] - \mu^2 = \mathbb{E}[X^2] - (\mathbb{E}[X])^2. && \text{(simplify)} && (2.9)
\end{aligned}$$

### 2.4.3 Derivation 3: MLE for a Gaussian mean

Given $x_1, \ldots, x_n$ iid $\mathcal{N}(\mu, \sigma^2)$ with $\sigma^2$ known, find $\hat{\mu}_{\mathrm{MLE}}$.

The log-likelihood:

$$\log L(\mu) = \sum_{i=1}^{n} \log\left[\frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x_i-\mu)^2}{2\sigma^2}\right)\right] \qquad \text{(plug in pdf, take log)} \tag{2.10}$$

$$= -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i-\mu)^2. \qquad \text{(simplify)} \tag{2.11}$$

Differentiate with respect to $\mu$ and set to zero:

$$\frac{d}{d\mu}\log L = \frac{1}{\sigma^2}\sum_{i=1}^{n}(x_i-\mu) = 0 \qquad \text{(only second term depends on } \mu\text{)} \tag{2.12}$$

$$\sum_{i=1}^{n} x_i - n\mu = 0 \qquad \text{(multiply by } \sigma^2\text{)} \tag{2.13}$$

$$\hat{\mu}_{\mathrm{MLE}} = \frac{1}{n}\sum_{i=1}^{n} x_i. \qquad \text{(solve)} \tag{2.14}$$

The maximum-likelihood estimate is the sample mean. This explains why averaging is so natural: it is the optimal Gaussian-mean estimator.

### 2.4.4 Derivation 4: Linearity of expectation

For any random variables $X, Y$ (not necessarily independent) and any constants $a, b$,

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y].$$

The discrete proof:

$$\begin{aligned}
\mathbb{E}[aX + bY] &= \sum_{x,y} (ax + by)\, p(x, y) && \text{(definition of expectation)} && (2.15)\\
&= a \sum_{x,y} x\, p(x, y) + b \sum_{x,y} y\, p(x, y) && \text{(distribute)} && (2.16)\\
&= a \sum_{x} x \sum_{y} p(x, y) + b \sum_{y} y \sum_{x} p(x, y) && \text{(reorder sums)} && (2.17)\\
&= a \sum_{x} x\, p(x) + b \sum_{y} y\, p(y) && \text{(marginal definitions)} && (2.18)\\
&= a\mathbb{E}[X] + b\mathbb{E}[Y]. && \text{(definition)} && (2.19)
\end{aligned}$$

Note where independence is *not* used: anywhere. Linearity of expectation always holds. This is why summing many random rewards or losses is so well-behaved.

### 2.4.5 Derivation 5: Variance of a Sum of Independent Variables

Linearity of expectation works for any variables. Variance is fussier: it adds nicely *only when variables are independent*. Here is the derivation, with careful tracking of where independence enters.

Let $X$ and $Y$ be random variables with means $\mu_X = \mathbb{E}[X]$ and $\mu_Y = \mathbb{E}[Y]$. By definition,

$$\begin{aligned}
\mathrm{Var}(X + Y) &= \mathbb{E}\big[((X + Y) - \mathbb{E}[X + Y])^2\big] && \text{definition of variance} && (2.20)\\
&= \mathbb{E}\big[((X - \mu_X) + (Y - \mu_Y))^2\big] && \text{linearity of } \mathbb{E} && (2.21)\\
&= \mathbb{E}\big[(X - \mu_X)^2 + 2(X - \mu_X)(Y - \mu_Y) + (Y - \mu_Y)^2\big] && \text{expand square} && (2.22)\\
&= \mathbb{E}[(X - \mu_X)^2] + 2\,\mathbb{E}[(X - \mu_X)(Y - \mu_Y)] + \mathbb{E}[(Y - \mu_Y)^2] && \text{linearity} && (2.23)\\
&= \mathrm{Var}(X) + 2\,\mathrm{Cov}(X, Y) + \mathrm{Var}(Y) && \text{by definition.} && (2.24)
\end{aligned}$$

That last identity, $\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\mathrm{Cov}(X, Y)$, holds for any two random variables. Now suppose $X$ and $Y$ are independent. Independence implies $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$, so

$$\begin{aligned}
\mathrm{Cov}(X, Y) &= \mathbb{E}[(X - \mu_X)(Y - \mu_Y)] && && (2.25)\\
&= \mathbb{E}[XY] - \mu_X\mathbb{E}[Y] - \mu_Y\mathbb{E}[X] + \mu_X\mu_Y && \text{expand} && (2.26)\\
&= \mu_X\mu_Y - \mu_X\mu_Y - \mu_Y\mu_X + \mu_X\mu_Y && \text{independence: } \mathbb{E}[XY] = \mu_X\mu_Y && (2.27)\\
&= 0. && && (2.28)
\end{aligned}$$

The cross term vanishes, leaving

$$\boxed{\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) \qquad \text{(when } X \perp Y\text{).}}$$

**Why this matters in machine learning.** When you average $n$ independent measurements with variance $\sigma^2$ each,

$$\mathrm{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \frac{1}{n^2}\sum_{i=1}^{n}\mathrm{Var}(X_i) = \frac{\sigma^2}{n}.$$

The standard deviation of the average shrinks as $\sigma/\sqrt{n}$. This is the law of large numbers in quantitative form, and it is why mini-batch gradient estimates become more reliable (linearly in batch size, in variance) but with diminishing returns (square-root in standard error).

## 2.5 Worked Examples

**Example 2.1: Basic Bayes**

A medical test for a disease has sensitivity 99% and specificity 95%. The disease prevalence is 1%. A patient tests positive. What is the probability they have the disease?
**Solution.** Let $D$ = "has disease", $T$ = "positive test". Given:

$$P(T \mid D) = 0.99, \quad P(T \mid D^c) = 0.05, \quad P(D) = 0.01.$$

Total probability of a positive test:

$$P(T) = P(T \mid D)P(D) + P(T \mid D^c)P(D^c) = (0.99)(0.01) + (0.05)(0.99) = 0.0099 + 0.0495 = 0.0594.$$

By Bayes:

$$P(D \mid T) = \frac{(0.99)(0.01)}{0.0594} = \frac{0.0099}{0.0594} \approx 0.167.$$

Only about 16.7%. This is the famous base-rate counter-intuitive result.

**Example 2.2: Expectation of a die**

Compute the expected value of a fair six-sided die roll.
**Solution.**

$$\mathbb{E}[X] = \sum_{k=1}^{6} k \cdot \frac{1}{6} = \frac{1+2+3+4+5+6}{6} = \frac{21}{6} = 3.5.$$

**Example 2.3: Variance of a Bernoulli**

Compute $\mathrm{Var}(X)$ for $X \sim \mathrm{Bernoulli}(p)$.
**Solution.**

$$\begin{aligned} \mathbb{E}[X] &= 1 \cdot p + 0 \cdot (1-p) = p, \\ \mathbb{E}[X^2] &= 1^2 \cdot p + 0^2 \cdot (1-p) = p, \\ \mathrm{Var}(X) &= \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = p - p^2 = p(1-p). \end{aligned}$$

**Example 2.4: Binomial probability**

You flip a fair coin 5 times. What is the probability of exactly 3 heads?
**Solution.**

$$P(X=3) = \binom{5}{3}(0.5)^3(0.5)^2 = 10 \cdot \frac{1}{8} \cdot \frac{1}{4} = \frac{10}{32} = 0.3125.$$

**Example 2.5: MLE for Bernoulli**

You observe a sequence of 10 coin flips: 7 heads, 3 tails. What is the MLE of the bias $p$?
**Solution.** The log-likelihood is

$$\log L(p) = 7 \log p + 3 \log(1-p).$$

Differentiate:

$$\frac{d}{dp} \log L = \frac{7}{p} - \frac{3}{1-p} = 0 \implies 7(1-p) = 3p \implies p = 0.7.$$

The MLE is the empirical frequency: 7/10.

## 2.6 Real-World AI Applications

**Naive Bayes spam filters.** Treating email words as conditionally independent given spam status, the filter computes $\log P(\text{spam} \mid \text{email}) - \log P(\text{ham} \mid \text{email})$ as a sum of word log-probabilities. Despite the unrealistic independence assumption, naive Bayes performs astonishingly well.

**Cross-entropy loss.** The loss minimized by every classifier you have ever trained is the negative log likelihood under a Bernoulli or Categorical model. Training a softmax classifier with cross-entropy is exactly MLE.

**Variational Autoencoders.** VAEs assume data is generated from a latent Gaussian, then trained to maximize a lower bound on the log-likelihood. The KL divergence between the encoder distribution and a prior Gaussian is the regularizer.

**Bayesian neural networks.** Instead of point-estimating the weights, BNNs maintain a posterior distribution over weights. Predictions average over many sampled networks, yielding calibrated uncertainty estimates.

## 2.7 Visual Explanation

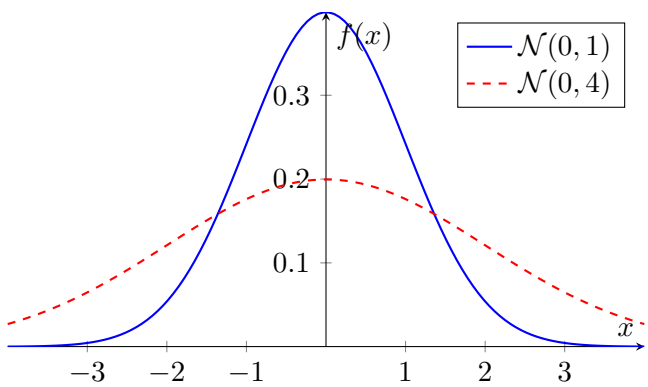



Figure 2.1: Two Gaussian densities. Larger variance flattens and widens the curve.

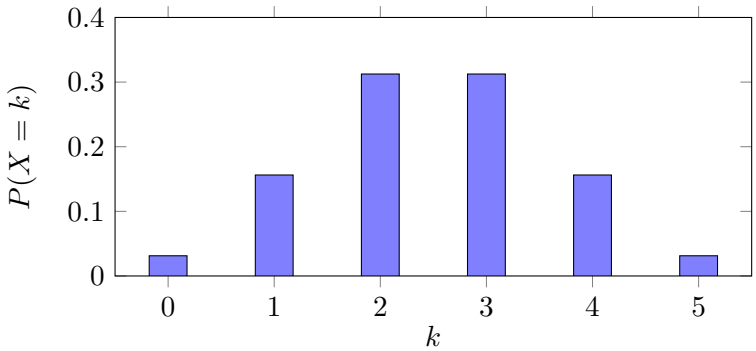



Figure 2.2: The Binomial$(5, 0.5)$ probability mass function.

## 2.8 Common Pitfalls

**Pitfall 1: Confusing $P(A \mid B)$ with $P(B \mid A)$.** In medical-test problems this is the source of nearly all wrong answers. The probability that you have a disease given a positive test is not the test's accuracy.

**Pitfall 2: Forgetting to normalize.** A probability distribution must sum (or integrate) to 1. When you write $P(X = k) \propto \cdots$, you still owe a normalizer.

**Pitfall 3: Treating variance as if it has the same units as $X$.** It does not. Variance has units of $X^2$. Use standard deviation if you want the same units.

**Pitfall 4: Believing independent and uncorrelated are the same.** Independence implies uncorrelated, but not vice versa. Two variables can have zero correlation and still be tightly coupled.

**Pitfall 5: Maximizing likelihood without log.** Products of probabilities underflow to zero in floating point. Always work with the log-likelihood.

**Pitfall 6: Confusing MLE with MAP.** They differ by the prior term. With a flat prior they coincide; with a strong prior they can disagree dramatically.

## 2.9 Chapter Summary

- Conditional probability: $P(A \mid B) = P(A \cap B)/P(B)$.
- Bayes:
$$\boxed{P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.}$$
- Expectation: $\mathbb{E}[X] = \sum_x xp(x)$ or $\int xf(x)\,dx$.
- Variance: $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$.
- Bernoulli mean $p$, var $p(1-p)$. Binomial mean $np$, var $np(1-p)$. Gaussian density $\propto e^{-(x-\mu)^2/(2\sigma^2)}$.
- CLT: averages tend to Gaussian.
- MLE picks $\theta$ maximizing $\sum \log p(x_i; \theta)$. MAP adds a prior log-term.

## 2.10 Homework

**Problem 1.** **[Easy]** A bag has 3 red and 7 blue balls. You draw one. Probability of red?

**Problem 2.** **[Easy]** If $X \sim \text{Bernoulli}(0.3)$, compute $\mathbb{E}[X]$ and $\text{Var}(X)$.

**Problem 3.** **[Easy]** Compute $P(X = 2)$ for $X \sim \text{Binomial}(4, 0.5)$.

**Problem 4.** **[Medium]** Two events satisfy $P(A) = 0.4$, $P(B) = 0.5$, $P(A \cap B) = 0.2$. Compute $P(A \mid B)$ and decide whether $A, B$ are independent.

**Problem 5.** **[Medium]** For $X \sim \mathcal{N}(0, 1)$, find $P(-1 \leq X \leq 1)$ using the rule of thumb 68–95–99.7.

**Problem 6.** **[Medium, AI]** A binary classifier produces probability 0.8 for the positive class. The true label is positive. Compute the cross-entropy loss $-\log(0.8)$.

**Problem 7.** **[Medium, AI]** A naive Bayes spam filter sees an email with words $w_1, w_2, w_3$. Given $P(\text{spam}) = 0.4$, $P(w_i \mid \text{spam}) = 0.5$ for each, $P(w_i \mid \text{ham}) = 0.2$, compute $P(\text{spam} \mid w_1, w_2, w_3)$.

**Problem 8.** **[Hard]** Prove that the MLE of $\sigma^2$ for $\mathcal{N}(\mu, \sigma^2)$ with $\mu$ known is $\frac{1}{n}\sum(x_i - \mu)^2$.

**Problem 9.** **[Hard]** Show that for independent $X, Y$, $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$.

**Problem 10.** **[Hard, AI]** A logistic regression model outputs $\sigma(\mathbf{w}^\top \mathbf{x})$. Show that maximum-likelihood training of this model is equivalent to minimizing the cross-entropy loss.

# Chapter 3

# Calculus

## 3.1 Opening Hook

When a neural network learns, it does so by adjusting its millions of weights to reduce a loss. To know which way to nudge each weight, the optimizer needs the derivative of the loss with respect to that weight. Multiply across the network and you get the gradient, a vector with as many entries as there are parameters. The whole machinery of backpropagation is just the chain rule applied at scale. Calculus is the engine that turns data into learning.

## 3.2 Why This Matters for AI

**Gradients drive learning.** Every gradient-based optimizer (SGD, Adam, RMSprop) computes derivatives of a loss and steps against them.

**The chain rule is backpropagation.** Backprop is one repeated application of the chain rule, from output back to input.

**Activation functions are calculus objects.** Sigmoid, tanh, ReLU, softmax: their derivatives shape what gradients look like.

**Hessians inform second-order methods.** Newton-style and natural-gradient methods use the second derivative matrix.

**Integrals appear in probability.** Normalizing constants, evidence in Bayes, expectations of continuous variables: all integrals.

## 3.3 Core Concepts

### 3.3.1 Limits

**Definition 3.1** (Limit)**.** $\lim_{x \to a} f(x) = L$ means that $f(x)$ can be made arbitrarily close to $L$ by making $x$ sufficiently close to $a$ (but not equal).

In plain English: as you slide $x$ towards $a$, the value of $f$ slides towards $L$.

### 3.3.2 Derivatives

**Definition 3.2** (Derivative)**.** $f'(x) = \lim_{h \to 0} \frac{f(x+h)-f(x)}{h}$ when this limit exists.

The derivative at $x$ is the slope of the tangent line to the graph of $f$ at $x$. Or: the instantaneous rate of change.

A short table you should memorize:

| $f(x)$ | $f'(x)$ |
|---|---|
| $c$ (constant) | $0$ |
| $x^n$ | $nx^{n-1}$ |
| $e^x$ | $e^x$ |
| $\log x$ | $1/x$ |
| $\sin x$ | $\cos x$ |
| $\cos x$ | $-\sin x$ |
| $\sigma(x) = 1/(1+e^{-x})$ | $\sigma(x)(1-\sigma(x))$ |

Differentiation rules:

$$(f+g)' = f' + g' \tag{3.1}$$

$$(fg)' = f'g + fg' \quad \text{(product rule)} \tag{3.2}$$

$$\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2} \quad \text{(quotient rule)} \tag{3.3}$$

$$(f(g(x)))' = f'(g(x))\, g'(x) \quad \text{(chain rule)} \tag{3.4}$$

### 3.3.3 Partial derivatives and the gradient

When $f$ has several inputs, the partial derivative $\frac{\partial f}{\partial x_i}$ is the derivative with respect to $x_i$ holding all other inputs fixed.

**Definition 3.3** (Gradient)**.** For $f : \mathbb{R}^n \to \mathbb{R}$, the gradient is

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}.$$

The gradient points in the direction of steepest ascent. To minimize $f$, you walk in the direction $-\nabla f$.

### 3.3.4 Jacobian and Hessian

For a vector-valued function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian is the $m \times n$ matrix of all partial derivatives:

$$\mathbf{J}_{ij} = \frac{\partial f_i}{\partial x_j}.$$

For a scalar function $f : \mathbb{R}^n \to \mathbb{R}$, the Hessian is the $n \times n$ matrix of second partial derivatives:

$$\mathbf{H}_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}.$$

### 3.3.5 Chain rule for many variables

If $z = f(y_1, \ldots, y_m)$ and each $y_j = g_j(x_1, \ldots, x_n)$, then

$$\frac{\partial z}{\partial x_i} = \sum_{j=1}^{m} \frac{\partial z}{\partial y_j} \cdot \frac{\partial y_j}{\partial x_i}.$$

This is exactly the rule that backpropagation uses: walk from output to input, multiplying local derivatives along the way.

### 3.3.6 Taylor series

Near a point $a$, a smooth function can be written as

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2}(x-a)^2 + \frac{f'''(a)}{6}(x-a)^3 + \cdots$$

This justifies all linearizations and second-order optimization methods.

### 3.3.7 Integration intuition

Integration is the reverse of differentiation, and it computes an area under a curve:

$$\int_a^b f(x)\,dx = \text{net signed area between } y = f(x) \text{ and the } x\text{-axis on } [a,b].$$

The fundamental theorem of calculus connects the two: if $F' = f$, then $\int_a^b f(x)\,dx = F(b) - F(a)$.

## 3.4 Step-by-Step Derivations

### 3.4.1 Derivation 1: Derivative of $\sigma(x) = 1/(1+e^{-x})$

We use the chain rule and quotient manipulations:

$$\begin{aligned}
\sigma(x) &= (1+e^{-x})^{-1} && \text{(rewrite)} && (3.5)\\
\sigma'(x) &= -1 \cdot (1+e^{-x})^{-2} \cdot (-e^{-x}) && \text{(chain rule)} && (3.6)\\
&= \frac{e^{-x}}{(1+e^{-x})^2} && \text{(simplify signs)} && (3.7)\\
&= \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} && \text{(split fraction)} && (3.8)\\
&= \sigma(x) \cdot \frac{(1+e^{-x})-1}{1+e^{-x}} && \text{(rewrite numerator)} && (3.9)\\
&= \sigma(x) \cdot (1-\sigma(x)). && \text{(simplify)} && (3.10)
\end{aligned}$$

This neat form makes sigmoid units fast to compute and differentiate.

### 3.4.2 Derivation 2: Gradient of squared loss

Let $L(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$. We will show $\nabla L = \mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$.

Expand:

$$\begin{aligned}
L(\mathbf{w}) &= \tfrac{1}{2}(\mathbf{X}\mathbf{w} - \mathbf{y})^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) && \text{(definition of squared norm)} && (3.11)\\
&= \tfrac{1}{2}(\mathbf{w}^\top\mathbf{X}^\top\mathbf{X}\mathbf{w} - 2\mathbf{y}^\top\mathbf{X}\mathbf{w} + \mathbf{y}^\top\mathbf{y}) && \text{(expand)} && (3.12)
\end{aligned}$$

Differentiate using the rules $\nabla_{\mathbf{w}}(\mathbf{w}^\top\mathbf{A}\mathbf{w}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{w}$ and $\nabla_{\mathbf{w}}(\mathbf{b}^\top\mathbf{w}) = \mathbf{b}$:

$$\begin{aligned}
\nabla L(\mathbf{w}) &= \tfrac{1}{2}\big(2\mathbf{X}^\top\mathbf{X}\mathbf{w} - 2\mathbf{X}^\top\mathbf{y}\big) && \text{(apply gradient rules)} && (3.13)\\
&= \mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}). && \text{(factor)} && (3.14)
\end{aligned}$$

Setting $\nabla L = 0$ gives the normal equations $\mathbf{X}^\top\mathbf{X}\mathbf{w} = \mathbf{X}^\top\mathbf{y}$ of linear regression.

### 3.4.3 Derivation 3: Second-order Taylor expansion

Suppose $f$ is twice differentiable. Define $g(t) = f(a + t(x - a))$, so $g(0) = f(a)$, $g(1) = f(x)$. By single-variable Taylor's theorem,

$$g(1) = g(0) + g'(0) + \tfrac{1}{2}g''(\xi), \quad \xi \in (0, 1).$$

Compute the derivatives:

$$g'(t) = \nabla f(a + t(x - a))^\top (x - a), \tag{3.15}$$
$$g'(0) = \nabla f(a)^\top (x - a), \tag{3.16}$$
$$g''(t) = (x - a)^\top \mathbf{H}(a + t(x - a))(x - a), \tag{3.17}$$

hence

$$f(x) = f(a) + \nabla f(a)^\top (x - a) + \tfrac{1}{2}(x - a)^\top \mathbf{H}(\xi)(x - a).$$

The Taylor expansion to second order; this is exactly what Newton's method uses.

### Derivation 4: Newton's Method from a Quadratic Approximation

You have a function $f(x)$ and you want to find a root — a point where $f(x) = 0$. You know roughly where the root lives. How do you home in on it? The answer is to replace $f$ with its tangent line at your current guess, and then solve the easy linear problem instead.

Suppose your current guess is $x_n$ and the true root is $x^*$. Write the first-order Taylor expansion of $f$ around $x_n$:

$$f(x) \approx f(x_n) + f'(x_n)\,(x - x_n) \qquad \text{tangent line at } x_n. \tag{3.18}$$

You want the right-hand side to be zero (because that is where the line crosses the axis), so set

$$0 = f(x_n) + f'(x_n)\,(x_{n+1} - x_n) \qquad \text{define next guess } x_{n+1}, \tag{3.19}$$

$$f'(x_n)\,(x_{n+1} - x_n) = -f(x_n) \qquad \text{move term,} \tag{3.20}$$

$$x_{n+1} - x_n = -\frac{f(x_n)}{f'(x_n)} \qquad \text{divide by } f'(x_n) \neq 0, \tag{3.21}$$

$$\boxed{x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.} \qquad \text{Newton's update.} \tag{3.22}$$

Now apply the same idea to optimization. To minimize $g(x)$ you need a stationary point: $g'(x) = 0$. Treat $g'$ as the function whose root you want, and feed it into Newton's method:

$$x_{n+1} = x_n - \frac{g'(x_n)}{g''(x_n)}.$$

In multiple dimensions the derivative becomes the gradient and the second derivative becomes the Hessian, giving

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \mathbf{H}(\mathbf{x}_n)^{-1}\,\nabla g(\mathbf{x}_n).$$

This is full Newton's method. It converges quadratically near a minimum — the number of correct digits roughly doubles each step — but each step costs a Hessian inverse, which is why deep learning prefers cheaper first-order methods like SGD.

### Derivation 5: Backpropagation Through a Two-Layer Network

This derivation is the chain rule put to work on the network you'll meet in your first deep-learning course. Hold the picture in your head: input $\mathbf{x}$, weights $\mathbf{W}_1$, hidden activation $\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x})$, weights $\mathbf{W}_2$, output $\hat{y} = \mathbf{W}_2\mathbf{h}$, scalar loss $L = \frac{1}{2}(\hat{y} - y)^2$.

Computing $\partial L/\partial \mathbf{W}_2$ is the easy direction; the gradient flows back one layer.

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y \qquad \text{derivative of } \tfrac{1}{2}(\hat{y} - y)^2, \tag{3.23}$$

$$\frac{\partial L}{\partial \mathbf{W}_2} = \frac{\partial L}{\partial \hat{y}} \cdot \mathbf{h}^\top = (\hat{y} - y)\,\mathbf{h}^\top \qquad \text{since } \hat{y} = \mathbf{W}_2\mathbf{h}. \tag{3.24}$$

That is the gradient with respect to the output-layer weights. Now flow back another layer to reach $\mathbf{W}_1$. Define the pre-activation $\mathbf{z} = \mathbf{W}_1\mathbf{x}$ and remember $\mathbf{h} = \sigma(\mathbf{z})$. The chain rule pieces are:

$$\frac{\partial L}{\partial \mathbf{h}} = \mathbf{W}_2^\top(\hat{y} - y) \qquad \text{transpose because output is row of } \mathbf{W}_2\mathbf{h}, \tag{3.25}$$

$$\frac{\partial \mathbf{h}}{\partial \mathbf{z}} = \operatorname{diag}(\sigma'(\mathbf{z})) \qquad \text{element-wise activation}, \tag{3.26}$$

$$\frac{\partial L}{\partial \mathbf{z}} = \operatorname{diag}(\sigma'(\mathbf{z}))\,\mathbf{W}_2^\top(\hat{y} - y) \qquad \text{compose} \tag{3.27}$$

$$= \sigma'(\mathbf{z}) \odot \left[\mathbf{W}_2^\top(\hat{y} - y)\right] \qquad \odot \text{ is element-wise multiply}, \tag{3.28}$$

$$\frac{\partial L}{\partial \mathbf{W}_1} = \frac{\partial L}{\partial \mathbf{z}}\,\mathbf{x}^\top \qquad \text{since } \mathbf{z} = \mathbf{W}_1\mathbf{x}. \tag{3.29}$$

Look at the structure: at each layer, the gradient with respect to a weight matrix is (the local error signal) times (the input that fed into that layer), transposed. The error signal itself is propagated backward by multiplying by the transposed weights of the next layer and by the local activation derivative. That recipe — forward pass to compute activations, backward pass to compute gradients — is backpropagation in its entirety. Every modern deep learning library does exactly this, just for many more layers and with vectorized matrix operations.

## 3.5 Worked Examples

**Example 3.1: Derivative of a polynomial**

Find $f'(x)$ if $f(x) = 3x^4 - 5x^2 + 7$.
**Solution.**

$$f'(x) = 12x^3 - 10x.$$

**Example 3.2: Chain rule**

Find $f'(x)$ if $f(x) = (2x + 1)^5$.
**Solution.** Let $u = 2x + 1$. Then $f = u^5$, $\frac{df}{du} = 5u^4$, $\frac{du}{dx} = 2$. So

$$f'(x) = 5u^4 \cdot 2 = 10(2x + 1)^4.$$

**Example 3.3: Partial derivatives**

For $f(x, y) = x^2y + 3xy^3$, compute both partials.
**Solution.**

$$\frac{\partial f}{\partial x} = 2xy + 3y^3,$$
$$\frac{\partial f}{\partial y} = x^2 + 9xy^2.$$

**Example 3.4: Gradient at a point**

For $f(x, y) = x^2 + xy + y^2$, compute $\nabla f$ at $(1, 2)$.
**Solution.**

$$\nabla f = (2x + y,\ x + 2y).$$

At $(1, 2)$: $\nabla f = (2 + 2,\ 1 + 4) = (4, 5)$.

**Example 3.5: Backprop sketch**

Suppose $z = (xw + b)^2$ with $x = 2$, $w = 3$, $b = 1$. Compute $\partial z/\partial w$ and $\partial z/\partial b$.
**Solution.** Let $a = xw + b = 7$. Then $z = a^2 = 49$. By chain rule:

$$\frac{\partial z}{\partial a} = 2a = 14,$$
$$\frac{\partial a}{\partial w} = x = 2, \quad \frac{\partial a}{\partial b} = 1,$$
$$\frac{\partial z}{\partial w} = 14 \cdot 2 = 28,$$
$$\frac{\partial z}{\partial b} = 14 \cdot 1 = 14.$$

## 3.6 Real-World AI Applications

**Backpropagation.** A modern neural-network forward pass is a directed acyclic graph of operations. Backprop walks the graph in reverse, applying the chain rule at each node. Frameworks like PyTorch and JAX automate this with reverse-mode automatic differentiation.

**Adam optimizer.** Adam tracks an exponentially decaying average of gradients and squared gradients, then takes step sizes that adapt per-parameter. Each component is a derivative.

**Softmax + cross-entropy.** The combined gradient of softmax composed with cross-entropy is just $\hat{\mathbf{y}} - \mathbf{y}$ (predicted minus true probabilities). This deceptively simple form is the result of careful chain-rule cancellation.

**Diffusion models.** Score-based diffusion models learn $\nabla_{\mathbf{x}} \log p(\mathbf{x})$, the gradient of the log-density. They generate images by following these gradients backward through noise.

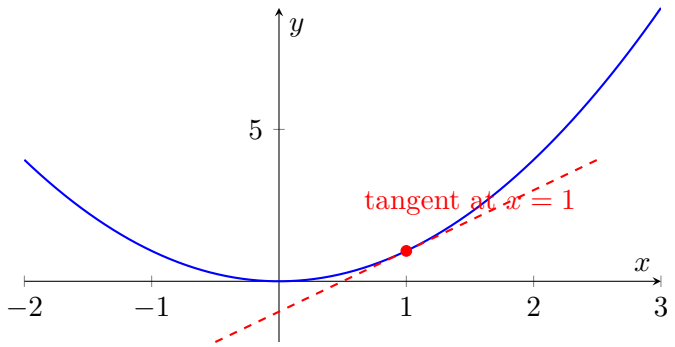



Figure 3.1: The tangent line at $x = 1$ has slope $f'(1) = 2$.

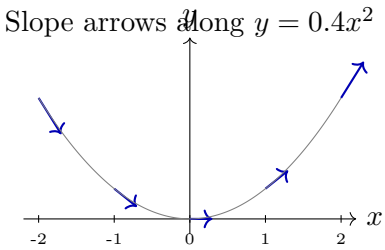



Figure 3.2: Slopes of $y = 0.4x^2$ at sample points; the derivative is the local slope.

## 3.7 Visual Explanation

## 3.8 Common Pitfalls

**Pitfall 1: Forgetting the chain rule.** Students differentiate $\sin(x^2)$ as $\cos(x^2)$. The correct answer is $2x\cos(x^2)$.

**Pitfall 2: Treating partial derivatives as full derivatives.** When you take $\partial f/\partial x$, every other variable is a constant, even if it "looks like" a function of $x$ in a different context.

**Pitfall 3: Sign errors in gradient descent.** You move in the direction of $-\nabla f$ to minimize, $+\nabla f$ to maximize. A sign flip is a common bug.

**Pitfall 4: Vanishing gradients with sigmoids.** $\sigma(x)(1-\sigma(x))$ peaks at 0.25 and decays to 0 for large $|x|$. Stacking many sigmoids destroys gradient signal.

**Pitfall 5: Confusing the Jacobian shape.** For $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian is $m \times n$. Many students transpose it without realizing.

**Pitfall 6: Differentiating ReLU at zero.** ReLU's derivative does not exist at 0. In practice it is set to 0 or 1 by convention; the chosen value rarely matters because the event has measure zero.

## 3.9 Chapter Summary

- Derivative: $f'(x) = \lim_{h\to 0}(f(x+h) - f(x))/h$.
- Chain rule: $(f \circ g)'(x) = f'(g(x))g'(x)$.
- Gradient: $\nabla f$ points in steepest ascent.
- Jacobian for vector-valued functions; Hessian for second derivatives.

- Sigmoid derivative:

$$\boxed{\sigma'(x) = \sigma(x)(1 - \sigma(x)).}$$

- Taylor series approximates $f$ near a point; basis for Newton's method.
- Integration is anti-differentiation; computes signed area.

## 3.10 Homework

**Problem 1.** **[Easy]** Differentiate $f(x) = 5x^3 - 4x + 2$.

**Problem 2.** **[Easy]** Find $\frac{d}{dx}\sin(3x)$.

**Problem 3.** **[Easy]** Compute $\frac{\partial}{\partial y}(x^2 + xy + y^2)$.

**Problem 4.** **[Medium]** Differentiate $f(x) = e^{-x^2}$.

**Problem 5.** **[Medium]** Compute the gradient of $f(x, y, z) = x^2 + 2y^3 - 4z$.

**Problem 6.** **[Medium, AI]** Show that $\frac{d}{dx}\log\sigma(x) = 1 - \sigma(x)$.

**Problem 7.** **[Medium, AI]** A neuron computes $a = \tanh(wx + b)$. Compute $\partial a/\partial w$ at $w = 0.5, x = 2, b = 0$. Use $\tanh'(u) = 1 - \tanh^2(u)$.

**Problem 8.** **[Hard]** Find the Hessian of $f(x, y) = x^2 + 3xy + 2y^2$ and decide if the origin is a min, max, or saddle.

**Problem 9.** **[Hard]** Show that the gradient of $f(\mathbf{w}) = \frac{1}{2}\|\mathbf{w}\|^2$ is $\mathbf{w}$.

**Problem 10.** **[Hard, AI]** Derive the gradient of cross-entropy loss with respect to the logits, given $L(\mathbf{z}) = -\sum_i y_i \log \frac{e^{z_i}}{\sum_j e^{z_j}}$. Show it equals $\hat{\mathbf{y}} - \mathbf{y}$ where $\hat{y}_i = e^{z_i}/\sum_j e^{z_j}$.

# Chapter 4

# Optimization

## 4.1 Opening Hook

A modern neural network has hundreds of millions of parameters. There is no closed-form formula that picks the best ones. Instead, training is an iterative descent: start with random weights, compute the gradient of the loss, take a small step opposite the gradient, repeat. The art of optimization is making this descent fast, stable, and reliable. Every clever trick in the deep-learning toolbox–momentum, Adam, learning-rate schedules, weight decay–is a small twist on the same idea.

## 4.2 Why This Matters for AI

**Training is optimization.** Every model you have heard of is the result of solving an optimization problem. Knowing the landscape determines what is solvable.

**Convexity is a guarantee.** For convex problems, gradient descent converges to the global minimum. For non-convex deep nets, we settle for good local minima.

**Learning rate is the most important hyperparameter.** Too large: the optimizer diverges. Too small: it crawls. Schedules and adaptive methods address this.

**Constrained optimization explains many models.** Support vector machines, sparse coding, and reinforcement-learning policies all use constraints solved by Lagrangian methods.

**Stochastic gradient is special.** Using mini-batches instead of the full dataset is not just a speedup; the noise can help escape saddle points.

## 4.3 Core Concepts

### 4.3.1 Convexity

**Definition 4.1** (Convex function)**.** A function $f : \mathbb{R}^n \to \mathbb{R}$ is convex if for all $\mathbf{x}, \mathbf{y}$ and $t \in [0, 1]$,

$$f(t\mathbf{x} + (1-t)\mathbf{y}) \leq tf(\mathbf{x}) + (1-t)f(\mathbf{y}).$$

In plain English: any chord of the graph lies above the graph itself. A bowl-shape.

For twice-differentiable $f$, convexity is equivalent to the Hessian being positive semidefinite everywhere.

**Important**

For convex $f$, every local minimum is a global minimum. This is the single most useful fact in optimization.

### 4.3.2 Gradient descent

The basic update rule:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla f(\mathbf{w}_t),$$

where $\eta > 0$ is the learning rate.

### 4.3.3 Momentum

Vanilla GD can oscillate in narrow valleys. Momentum smooths the path:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \nabla f(\mathbf{w}_t), \tag{4.1}$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{v}_{t+1}. \tag{4.2}$$

The velocity $\mathbf{v}$ accumulates past gradients with decay $\beta \in [0, 1)$.

### 4.3.4 Adam (intuition)

Adam tracks two moving averages: of the gradient ($\mathbf{m}$) and of its square ($\mathbf{v}$). The update divides by $\sqrt{\mathbf{v}}$ to scale per-parameter:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla f(\mathbf{w}_{t-1}), \tag{4.3}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)(\nabla f(\mathbf{w}_{t-1}))^2, \tag{4.4}$$

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}, \tag{4.5}$$

where $\hat{\mathbf{m}}_t, \hat{\mathbf{v}}_t$ are bias-corrected. Each parameter gets its own effective learning rate.

### 4.3.5 Stochastic vs batch

**Batch GD:** compute $\nabla f$ on the entire dataset; one slow, accurate step.

**SGD:** compute gradient on one example; many fast, noisy steps.

**Mini-batch SGD:** use a random subset (e.g., 32, 256). The right balance: fast and stable.

### 4.3.6 Local vs global minima

A non-convex landscape has many local minima, plateaus, and saddle points. In high dimensions, saddle points outnumber local minima. Stochastic noise tends to push iterates off saddle points and into low-loss basins.

### 4.3.7 Lagrange multipliers

To minimize $f(\mathbf{x})$ subject to $g(\mathbf{x}) = 0$, define the Lagrangian

$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda g(\mathbf{x}),$$

and solve $\nabla_{\mathbf{x}} \mathcal{L} = 0$ alongside $g = 0$. The multiplier $\lambda$ is the rate at which the optimal value changes when the constraint is relaxed.

## 4.4 Step-by-Step Derivations

### 4.4.1 Derivation 1: GD on a quadratic converges when $\eta < 2/L$

Let $f(w) = \frac{1}{2}Lw^2$ with $L > 0$. Then $f'(w) = Lw$.

GD update: $w_{t+1} = w_t - \eta L w_t = (1 - \eta L)w_t$.

Iterating:

$$w_t = (1 - \eta L)^t w_0. \qquad \text{(geometric)} \tag{4.6}$$

The iterates converge to zero iff $|1 - \eta L| < 1$, i.e. $0 < \eta < 2/L$. The fastest convergence is at $\eta = 1/L$.

### 4.4.2 Derivation 2: First-order optimality of Lagrangian

Consider minimizing $f(\mathbf{x})$ subject to $g(\mathbf{x}) = 0$. At a constrained optimum $\mathbf{x}^*$, the gradient $\nabla f(\mathbf{x}^*)$ must be parallel to $\nabla g(\mathbf{x}^*)$ (otherwise we could move along the constraint surface and decrease $f$). So there is a $\lambda$ with

$$\nabla f(\mathbf{x}^*) = \lambda \nabla g(\mathbf{x}^*),$$

which is the same as $\nabla_{\mathbf{x}} \mathcal{L} = \nabla f - \lambda \nabla g = 0$.

### 4.4.3 Derivation 3: Convexity of squared loss

Let $f(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$. Then

$$f(\mathbf{w}) = \mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} - 2\mathbf{y}^\top \mathbf{X}\mathbf{w} + \mathbf{y}^\top \mathbf{y}. \tag{4.7}$$

The Hessian is $2\mathbf{X}^\top \mathbf{X}$, which is positive semidefinite for any $\mathbf{X}$ (since $\mathbf{u}^\top \mathbf{X}^\top \mathbf{X}\mathbf{u} = \|\mathbf{X}\mathbf{u}\|^2 \geq 0$). Therefore $f$ is convex, and any minimum is global.

### 4.4.4 Derivation 4: Why Momentum Helps in a Narrow Valley

Imagine a long, narrow ravine: gentle slope along its length, steep walls along its width. Vanilla gradient descent oscillates back and forth across the walls and crawls along the floor. Momentum fixes this. Here's why.

The momentum update is

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \nabla f(\mathbf{x}_t) \qquad \text{accumulate gradient,} \tag{4.8}$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \mathbf{v}_{t+1} \qquad \text{step in velocity direction.} \tag{4.9}$$

Unroll the velocity recursion assuming we start from rest ($\mathbf{v}_0 = \mathbf{0}$):

$$\mathbf{v}_1 = \nabla f(\mathbf{x}_0), \tag{4.10}$$

$$\mathbf{v}_2 = \beta \nabla f(\mathbf{x}_0) + \nabla f(\mathbf{x}_1), \tag{4.11}$$

$$\mathbf{v}_3 = \beta^2 \nabla f(\mathbf{x}_0) + \beta \nabla f(\mathbf{x}_1) + \nabla f(\mathbf{x}_2), \tag{4.12}$$

$$\mathbf{v}_t = \sum_{k=0}^{t-1} \beta^{t-1-k} \nabla f(\mathbf{x}_k) \qquad \text{exponentially weighted sum.} \tag{4.13}$$

The current velocity is a weighted average of past gradients, with older ones decayed by $\beta^k$.

Now consider the two directions of the ravine separately. *Along the floor*, every gradient points the same way (gently downhill). The weighted sum reinforces itself, so velocity grows toward an effective magnitude

$$\|\mathbf{v}_t\| \to \frac{\|\nabla f\|}{1-\beta} \qquad \text{(geometric series sum).}$$

With $\beta = 0.9$, that is $1/(1-0.9) = 10\times$ amplification. *Across the walls*, gradients alternate sign each step. In the weighted sum, consecutive terms cancel, and net velocity stays small. So momentum acts as a low-pass filter: it amplifies the consistent low-frequency descent direction and dampens the high-frequency oscillation. That is why the same learning rate that diverged with vanilla GD becomes stable, and faster, with momentum.

### 4.4.5 Derivation 5: The Optimal Learning Rate for a Quadratic Bowl

Suppose you are minimizing $f(w) = \frac{1}{2}a(w-w^*)^2$ with $a > 0$. The minimum is at $w^*$. Gradient descent says

$$w_{t+1} = w_t - \eta f'(w_t) = w_t - \eta a(w_t - w^*). \tag{4.14}$$

Subtract $w^*$ from both sides to track the error $e_t = w_t - w^*$:

$$e_{t+1} = e_t - \eta a\, e_t = (1-\eta a)\, e_t. \tag{4.15}$$

After $t$ steps, $e_t = (1-\eta a)^t\, e_0$. The error shrinks geometrically with ratio $|1-\eta a|$, so we converge if and only if $|1-\eta a| < 1$, i.e.

$$0 < \eta < \frac{2}{a}.$$

The fastest convergence comes from making $|1-\eta a|$ as small as possible. The minimum is exactly zero, achieved at $\eta^* = 1/a$. So the optimal learning rate equals the reciprocal of the curvature.

What about higher dimensions? If $f(\mathbf{w}) = \frac{1}{2}\mathbf{w}^\top \mathbf{A}\mathbf{w}$ with symmetric positive-definite $\mathbf{A}$, decompose $\mathbf{A}$ in its eigenbasis. Each eigendirection behaves independently like the scalar case above, with curvature equal to the eigenvalue $\lambda_i$. A single learning rate $\eta$ has to work for all of them. Stability across all directions requires $\eta < 2/\lambda_{\max}$, where $\lambda_{\max}$ is the largest eigenvalue. The smallest direction $\lambda_{\min}$ converges with ratio $|1-\eta\lambda_{\min}|$. The best uniform choice is $\eta = 2/(\lambda_{\max} + \lambda_{\min})$, giving convergence ratio

$$\rho = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa - 1}{\kappa + 1}, \qquad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

The number $\kappa$ is the *condition number*. When $\kappa \approx 1$, $\rho \approx 0$ and convergence is fast. When $\kappa$ is huge (badly conditioned), $\rho \approx 1$ and convergence is glacially slow. This is the formal explanation of why narrow ravines (large $\kappa$) defeat vanilla gradient descent — and why preconditioners, second-order methods, and adaptive optimizers like Adam exist.

## 4.5 Worked Examples

**Example 4.1: GD by hand**

Minimize $f(w) = (w-3)^2$ starting at $w_0 = 0$ with $\eta = 0.1$.
**Solution.** $f'(w) = 2(w-3)$. Updates:

$$\begin{aligned} w_1 &= 0 - 0.1 \cdot (-6) = 0.6 \\ w_2 &= 0.6 - 0.1 \cdot 2(0.6-3) = 0.6 - 0.1 \cdot (-4.8) = 1.08 \\ w_3 &= 1.08 - 0.1 \cdot 2(-1.92) = 1.08 + 0.384 = 1.464. \end{aligned}$$

Each step shrinks the distance to 3 by a factor 0.8.

**Example 4.2: Effect of large learning rate**

Same problem, $\eta = 1$.
**Solution.**

$$\begin{aligned} w_1 &= 0 - 1 \cdot (-6) = 6, \\ w_2 &= 6 - 1 \cdot 2(3) = 0, \\ w_3 &= 0 - 1 \cdot (-6) = 6, \ldots \end{aligned}$$

The iterates oscillate forever. Choose $\eta = 2/L = 2/2 = 1$ and you sit at the boundary of stability.

**Example 4.3: Constrained minimization**

Minimize $f(x, y) = x^2 + y^2$ subject to $x + y = 1$.
**Solution.** Lagrangian $\mathcal{L} = x^2 + y^2 - \lambda(x + y - 1)$. Stationarity:

$$2x = \lambda, \quad 2y = \lambda, \quad x + y = 1.$$

So $x = y = \lambda/2$, giving $\lambda/2 + \lambda/2 = 1$, $\lambda = 1$, $x = y = 1/2$. Minimum value $1/2$.

**Example 4.4: Convexity check**

Is $f(x) = x^4 - 2x^2$ convex?
**Solution.** $f''(x) = 12x^2 - 4$. This is negative when $|x| < 1/\sqrt{3}$, so $f$ is not globally convex.

**Example 4.5: SGD vs GD**

For $f(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(\mathbf{w}^\top \mathbf{x}_i - y_i)^2$, write the GD and SGD updates.
**Solution.**

$$\begin{aligned} \text{GD:} \quad & \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \cdot \frac{2}{n}\sum_i (\mathbf{w}_t^\top \mathbf{x}_i - y_i)\mathbf{x}_i. \\ \text{SGD:} \quad & \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \cdot 2(\mathbf{w}_t^\top \mathbf{x}_{i_t} - y_{i_t})\mathbf{x}_{i_t}, \quad i_t \text{ random.} \end{aligned}$$

Each SGD step is $1/n$-th the cost of GD but uses a noisy gradient estimate.

## 4.6 Real-World AI Applications

**Training transformers.** Modern LLMs use AdamW (Adam with decoupled weight decay) plus cosine learning-rate schedules. The optimizer is the difference between a model that trains in days and one that diverges in minutes.

**Support Vector Machines.** The SVM solves $\min \frac{1}{2}\|\mathbf{w}\|^2$ subject to $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$, a constrained convex problem solved with Lagrangian duality.

**Reinforcement learning policy gradients.** Policy gradient methods perform stochastic gradient ascent on expected return; the gradient is estimated from sampled trajectories.

**Hyperparameter optimization.** Bayesian optimization, grid search, and random search are themselves optimization procedures, but over the meta-objective "validation loss as a function of hyperparameters".

## 4.7 Visual Explanation

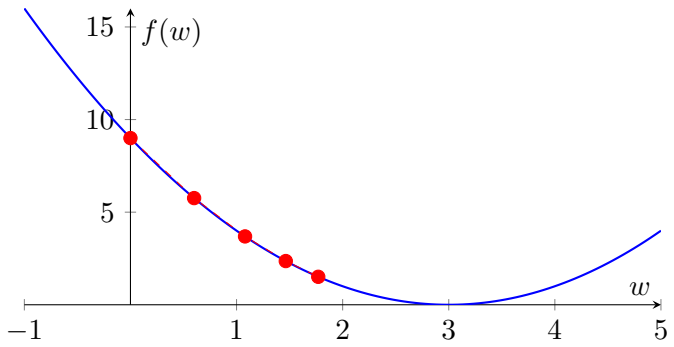



Figure 4.1: Gradient descent on $f(w) = (w-3)^2$ with $\eta = 0.1$ converges towards the minimum at $w = 3$.

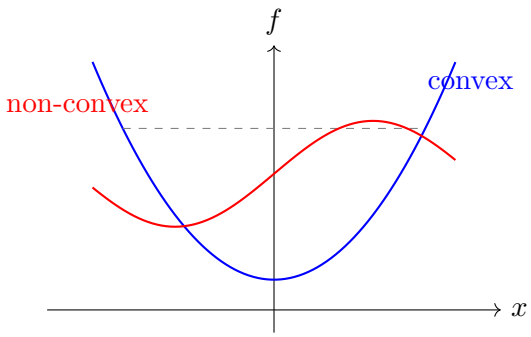



Figure 4.2: Convex (blue) versus non-convex (red): the convex bowl has a unique global minimum; the wavy curve has many local minima.

## 4.8 Common Pitfalls

**Pitfall 1: Learning rate too high.** The loss explodes to NaN. Reduce $\eta$ by $10\times$ until it stabilizes.

**Pitfall 2: Stuck at saddle points.** In flat regions, gradients are tiny and progress stops. Add momentum or use Adam.

**Pitfall 3: Forgetting to shuffle data.** SGD relies on random sampling. If you always feed examples in the same order, you can get stuck in cycles.

**Pitfall 4: Believing the loss curve uncritically.** A decreasing training loss does not mean the model generalizes. Always track validation loss too.

**Pitfall 5: Misuse of momentum.** Setting $\beta = 0.99$ on a noisy problem creates very long autocorrelation; iterates can overshoot. Default to $\beta = 0.9$.

**Pitfall 6: Lagrange multipliers without checking constraint qualification.** Lagrangian conditions are necessary, not always sufficient. Always check the second-order test or convexity.

## 4.9 Chapter Summary

- Convex functions have global minima.
- GD: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla f$.
- Momentum smooths oscillations; Adam scales per-parameter.
- SGD trades accuracy of gradient estimate for speed.
- Saddle points dominate high-dimensional non-convex problems.
- Lagrangian:

$$\boxed{\nabla f(\mathbf{x}^*) = \lambda \nabla g(\mathbf{x}^*) \text{ at constrained optimum.}}$$

## 4.10 Homework

**Problem 1.** **[Easy]** Determine whether $f(x) = x^4$ is convex.

**Problem 2.** **[Easy]** Perform two GD steps on $f(w) = w^2$ with $w_0 = 2$, $\eta = 0.3$.

**Problem 3.** **[Easy]** Find the minimum of $f(x) = x^2 - 4x + 7$ analytically.

**Problem 4.** **[Medium]** Find the largest learning rate $\eta$ such that GD on $f(w) = 4w^2$ converges.

**Problem 5.** **[Medium]** Use Lagrange multipliers to maximize $xy$ subject to $x + y = 10$.

**Problem 6.** **[Medium, AI]** A neural network has loss surface $f(w_1, w_2) = (w_1 - 1)^2 + 2(w_2 - 2)^2$. Starting at $(0, 0)$ with $\eta = 0.1$, compute three GD steps.

**Problem 7.** **[Medium, AI]** Why does Adam often outperform SGD on transformer training? Give two reasons related to gradient scale.

**Problem 8.** **[Hard]** Show that the sum of two convex functions is convex.

**Problem 9.** **[Hard]** Prove that for strictly convex $f$, gradient descent with fixed step size converges from any starting point if $\eta < 2/L$, where $L$ is the largest eigenvalue of the Hessian.

**Problem 10.** **[Hard, AI]** The hard-margin SVM solves $\min \frac{1}{2}\|\mathbf{w}\|^2$ subject to $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$. Form the Lagrangian and derive the dual problem in terms of $\alpha_i \geq 0$.

# Chapter 5

# Information Theory

## 5.1 Opening Hook

When you train a classifier with cross-entropy loss, you are minimizing exactly the same quantity that Claude Shannon defined in 1948 to quantify the information content of a message. When a generative model is trained with KL divergence, it is being told to make its distribution as indistinguishable as possible from the data distribution, in the precise sense that information theory provides. Information theory is the bridge between probability and the bit, and it shows up in every loss function in machine learning.

## 5.2 Why This Matters for AI

**Cross-entropy is the universal classification loss.** Every neural classifier you train minimizes cross-entropy.

**KL divergence regularizes generative models.** VAEs, normalizing flows, and reinforcement learning all use $D_{\mathrm{KL}}$.

**Mutual information measures dependence.** Feature selection, representation learning (InfoNCE, CLIP), and disentanglement all hinge on mutual information.

**Entropy as exploration bonus.** Maximum-entropy reinforcement learning encourages diverse policies by adding $H(\pi)$ to the reward.

**Why** log **is everywhere.** Probabilities multiply; log-probabilities add. That is why the additive structure of entropy is the natural one.

## 5.3 Core Concepts

### 5.3.1 Entropy

**Definition 5.1** (Entropy)**.** For a discrete random variable $X$ with pmf $p$,

$$H(X) = -\sum_x p(x) \log p(x).$$

In plain English: entropy is the average "surprise" of $X$. If $X$ is certain, $H = 0$; if $X$ is uniform over $n$ values, $H = \log n$.

The base of the log determines the unit. Base 2 gives bits, base $e$ gives nats. AI work usually uses base $e$.

### 5.3.2 Cross-entropy

**Definition 5.2** (Cross-entropy)**.** Between two distributions $p$ and $q$ on the same space,

$$H(p, q) = -\sum_x p(x) \log q(x).$$

The expected "surprise" if you assume the world is $q$ but the world is actually $p$. It is always at least $H(p)$.

### 5.3.3 KL divergence

**Definition 5.3** (Kullback–Leibler divergence)**.**

$$D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = H(p, q) - H(p).$$

KL is the gap between the cross-entropy and the entropy. It measures how far $q$ is from $p$. Three properties:

1. $D_{\mathrm{KL}}(p\|q) \geq 0$.
2. $D_{\mathrm{KL}}(p\|q) = 0$ iff $p = q$ (everywhere $p$ has support).
3. $D_{\mathrm{KL}}$ is not symmetric: $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$.

### 5.3.4 Mutual information

**Definition 5.4** (Mutual information)**.**

$$I(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} = D_{\mathrm{KL}}\big(p(x, y) \,\|\, p(x)p(y)\big).$$

Mutual information is the KL divergence between the joint distribution and the product of marginals. It is zero iff $X$ and $Y$ are independent. It quantifies how much knowing $Y$ reduces the uncertainty about $X$.

### 5.3.5 Why log appears everywhere

The choice of log is not arbitrary. The function $f(p) = -\log p$ is the unique (up to scale) function for which the information of independent events adds: $-\log(p_1 p_2) = -\log p_1 - \log p_2$. Multiplying probabilities corresponds to adding information. That is the whole story.

## 5.4 Step-by-Step Derivations

### 5.4.1 Derivation 1: $D_{\mathrm{KL}}(p\|q) \geq 0$

We use $\log x \leq x - 1$ for all $x > 0$, with equality iff $x = 1$.

$$
\begin{aligned}
-D_{\mathrm{KL}}(p\|q) &= \sum_x p(x)\log\frac{q(x)}{p(x)} && \text{(rewrite)} && (5.1)\\
&\le \sum_x p(x)\left(\frac{q(x)}{p(x)}-1\right) && \text{(log inequality)} && (5.2)\\
&= \sum_x q(x) - \sum_x p(x) && \text{(simplify)} && (5.3)\\
&= 1-1=0. && \text{(both distributions sum to 1)} && (5.4)
\end{aligned}
$$

Therefore $D_{\mathrm{KL}}(p\|q) \ge 0$, with equality iff $q(x)/p(x) = 1$ everywhere.

### 5.4.2 Derivation 2: Cross-entropy = entropy + KL

Direct expansion:

$$
\begin{aligned}
H(p,q) &= -\sum_x p(x)\log q(x) && \text{(definition)} && (5.5)\\
&= -\sum_x p(x)\log p(x) + \sum_x p(x)\log\frac{p(x)}{q(x)} && \text{(insert } \log p - \log p\text{)} && (5.6)\\
&= H(p) + D_{\mathrm{KL}}(p\|q). && \text{(definitions)} && (5.7)
\end{aligned}
$$

Minimizing cross-entropy with respect to $q$ is equivalent to minimizing $D_{\mathrm{KL}}(p\|q)$, since $H(p)$ does not depend on $q$.

### 5.4.3 Derivation 3: Mutual information in entropy form

$$
\begin{aligned}
I(X;Y) &= \sum_{x,y} p(x,y)\log\frac{p(x,y)}{p(x)p(y)} && \text{(definition)} && (5.8)\\
&= \sum_{x,y} p(x,y)\big[\log p(x,y) - \log p(x) - \log p(y)\big] && \text{(split log)} && (5.9)\\
&= -H(X,Y) - \sum_x \log p(x) \sum_y p(x,y)\,(-1)\cdots && && (5.10)\\
& && && (5.11)
\end{aligned}
$$

Reorganize step-by-step:

$$
\begin{aligned}
I(X;Y) &= -\sum_{x,y} p(x,y)\log\frac{1}{p(x,y)} - \sum_x p(x)\log p(x) - \sum_y p(y)\log p(y) && (5.12)\\
&= H(X) + H(Y) - H(X,Y). && (5.13)
\end{aligned}
$$

This is the form usually quoted in textbooks: information shared between $X$ and $Y$ is the sum of marginal entropies minus the joint entropy.

### 5.4.4 Derivation 4: The Beautifully Simple Softmax + Cross-Entropy Gradient

This derivation is the reason every classifier you'll ever train uses softmax with cross-entropy loss together. The pair has a gradient so clean it almost looks like a bug.

A network outputs raw scores (logits) $\mathbf{z} = (z_1, \ldots, z_K)$ for $K$ classes. Softmax converts them to probabilities:

$$
\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.
$$

The true label is one-hot: $\mathbf{y}$ has a 1 at the correct class index $c$ and zeros elsewhere. Cross-entropy loss is

$$L = -\sum_{i=1}^{K} y_i \log \hat{y}_i = -\log \hat{y}_c.$$

We want $\partial L/\partial z_k$ for every $k$. Step one: differentiate $L$ in terms of $\hat{y}$ and apply the chain rule:

$$\frac{\partial L}{\partial z_k} = -\sum_i y_i \frac{1}{\hat{y}_i} \frac{\partial \hat{y}_i}{\partial z_k} \qquad \text{chain rule.} \tag{5.14}$$

Step two: compute $\partial \hat{y}_i/\partial z_k$. Let $S = \sum_j e^{z_j}$. Then $\hat{y}_i = e^{z_i}/S$, so

$$\frac{\partial \hat{y}_i}{\partial z_k} = \frac{(\partial e^{z_i}/\partial z_k)\, S - e^{z_i}\,(\partial S/\partial z_k)}{S^2} \qquad \text{quotient rule} \tag{5.15}$$

$$= \frac{[\![i{=}k]\!]\, e^{z_i}\, S - e^{z_i} e^{z_k}}{S^2} \qquad \text{both derivatives are exponentials} \tag{5.16}$$

$$= [\![i{=}k]\!]\, \hat{y}_i - \hat{y}_i \hat{y}_k \qquad \text{factor and substitute.} \tag{5.17}$$

(Here $[\![\cdot]\!]$ is 1 if the condition holds, else 0.)

Step three: substitute back:

$$\frac{\partial L}{\partial z_k} = -\sum_i \frac{y_i}{\hat{y}_i}\Big([\![i{=}k]\!]\, \hat{y}_i - \hat{y}_i \hat{y}_k\Big) \qquad \text{plug in} \tag{5.18}$$

$$= -\sum_i y_i\big([\![i{=}k]\!] - \hat{y}_k\big) \qquad \text{cancel } \hat{y}_i \tag{5.19}$$

$$= -y_k + \hat{y}_k \sum_i y_i \qquad \text{distribute} \tag{5.20}$$

$$= \hat{y}_k - y_k \qquad \text{since } \sum_i y_i = 1. \tag{5.21}$$

That is the entire result: $\nabla_{\mathbf{z}} L = \hat{\mathbf{y}} - \mathbf{y}$. The gradient is just the predicted probabilities minus the true one-hot label. Every messy quotient rule and exponential cancels. This is why backpropagation through a softmax classifier is so cheap, and why nobody ever uses softmax with squared-error loss.

### 5.4.5 Derivation 5: The Differential Entropy of a Gaussian

For continuous distributions, the analog of entropy is *differential entropy*:

$$h(X) = -\int p(x) \log p(x)\, dx \quad \text{(in nats, using natural log).}$$

We will compute it for $X \sim \mathcal{N}(\mu, \sigma^2)$, where

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Take the log of the density:

$$\log p(x) = -\frac{1}{2}\log(2\pi\sigma^2) - \frac{(x-\mu)^2}{2\sigma^2} \qquad \text{log of product.} \tag{5.22}$$

Plug into the entropy integral and use linearity:

$$
\begin{aligned}
h(X) &= -\mathbb{E}[\log p(X)] && \text{rewrite as expectation} && (5.23)\\
&= \frac{1}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\,\mathbb{E}[(X-\mu)^2] && \text{linearity} && (5.24)\\
&= \frac{1}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\cdot\sigma^2 && \mathbb{E}[(X-\mu)^2] = \mathrm{Var}(X) = \sigma^2 && (5.25)\\
&= \frac{1}{2}\log(2\pi\sigma^2) + \frac{1}{2} && && (5.26)\\
&= \frac{1}{2}\log(2\pi e\sigma^2). && \text{absorb } \tfrac{1}{2} \text{ into log} && (5.27)
\end{aligned}
$$

Two consequences worth knowing. First, the entropy depends only on $\sigma$, not on $\mu$ — shifting a distribution doesn't change how uncertain it is, only spreading it does. Second, among all real-valued distributions with a given variance $\sigma^2$, the Gaussian *maximizes* differential entropy. This is why the central limit theorem makes Gaussians appear: averaging many independent variables increases entropy, and the Gaussian is the resting point. It is also why "add Gaussian noise" is the most informative-yet-most-honest noise model in any system constrained by power: nothing else with the same variance can hide more.

## 5.5 Worked Examples

**Example 5.1: Entropy of a fair coin**

Compute $H(X)$ for $X \sim \mathrm{Bernoulli}(0.5)$ in bits.
**Solution.**

$$H(X) = -0.5\log_2 0.5 - 0.5\log_2 0.5 = 0.5 + 0.5 = 1 \text{ bit.}$$

**Example 5.2: Entropy of a biased coin**

Compute $H(X)$ for $X \sim \mathrm{Bernoulli}(0.9)$ in bits.
**Solution.**

$$
\begin{aligned}
H(X) &= -0.9\log_2 0.9 - 0.1\log_2 0.1\\
&\approx -0.9\cdot(-0.152) - 0.1\cdot(-3.322)\\
&\approx 0.137 + 0.332 = 0.469 \text{ bits.}
\end{aligned}
$$

A biased coin carries less information than a fair one.

**Example 5.3: Cross-entropy classification loss**

A 3-class softmax outputs $\hat{y} = (0.7, 0.2, 0.1)$. The true label is class 1 (one-hot $y = (1, 0, 0)$). Compute the cross-entropy loss in nats.
**Solution.**

$$H(y, \hat{y}) = -1\cdot\log 0.7 - 0\cdot\log 0.2 - 0\cdot\log 0.1 = -\log 0.7 \approx 0.357.$$

**Example 5.4: KL between two Bernoullis**

Compute $D_{\mathrm{KL}}(p\|q)$ where $p = \mathrm{Bern}(0.6)$ and $q = \mathrm{Bern}(0.4)$ (in nats).

**Solution.**

$$\begin{aligned} D_{\mathrm{KL}}(p\|q) &= 0.6 \log \frac{0.6}{0.4} + 0.4 \log \frac{0.4}{0.6} \\ &= 0.6 \cdot 0.405 + 0.4 \cdot (-0.405) \\ &= 0.243 - 0.162 = 0.081 \text{ nats.} \end{aligned}$$

**Example 5.5: Mutual information of independent variables**

Show that $I(X;Y) = 0$ when $X$ and $Y$ are independent.
**Solution.** Independence means $p(x,y) = p(x)p(y)$. Then

$$I(X;Y) = \sum_{x,y} p(x)p(y) \log \frac{p(x)p(y)}{p(x)p(y)} = \sum_{x,y} p(x)p(y) \cdot 0 = 0.$$

## 5.6 Real-World AI Applications

**Cross-entropy training.** The de facto loss for image classifiers, language models, and most discriminative networks is the negative log-likelihood, which is exactly cross-entropy with respect to the data distribution.

**Variational Autoencoders.** The ELBO objective decomposes into a reconstruction term plus a $D_{\mathrm{KL}}$ regularizer that pulls the encoder distribution toward a prior.

**Contrastive learning (InfoNCE).** CLIP, SimCLR, and many self-supervised methods optimize a lower bound on mutual information between paired views.

**Reinforcement-learning policy regularization.** TRPO, PPO, and SAC all use KL divergence either as a trust-region constraint or as an exploration bonus.

## 5.7 Visual Explanation

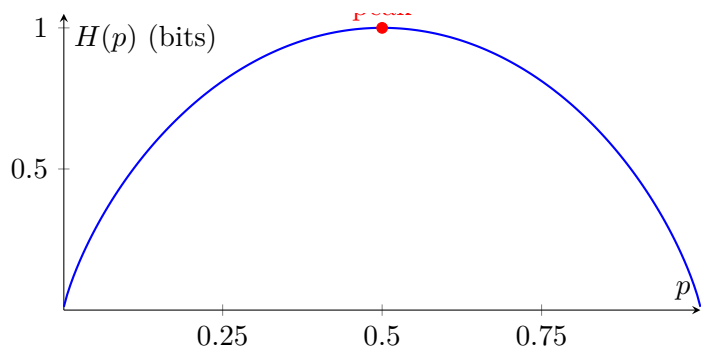



Figure 5.1: Binary entropy peaks at $p = 0.5$ (the fair coin) and is zero at $p \in \{0, 1\}$.

## 5.8 Common Pitfalls

**Pitfall 1: Treating KL as a distance.** It satisfies $D \geq 0$ and $D = 0$ iff equal, but not symmetry or the triangle inequality.

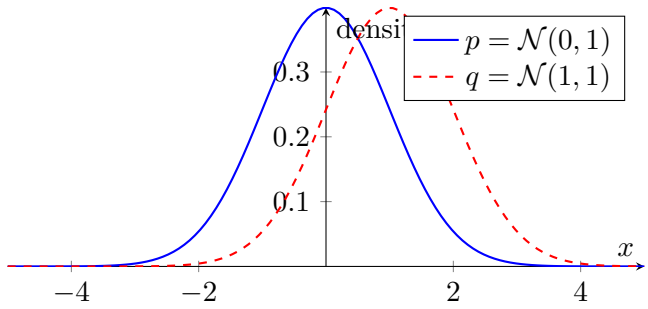



Figure 5.2: Two Gaussians with the same variance; their KL divergence is $\frac{1}{2} \cdot 1 = 0.5$ nats.

**Pitfall 2: Mixing log bases.** Bits use $\log_2$, nats use $\log_e$. The conversion is a multiplicative factor of $\log_2 e \approx 1.443$.

**Pitfall 3: Forgetting that $0 \log 0 = 0$.** By convention, this convention prevents NaN when probabilities are exactly zero.

**Pitfall 4: Computing cross-entropy with raw logits.** If you forget to apply softmax, you get nonsense. Most frameworks bundle them as "cross-entropy with logits" for stability.

**Pitfall 5: Mistaking high entropy for high information.** Entropy measures uncertainty, not utility. Pure noise has maximum entropy and zero useful information.

**Pitfall 6: Reading off mutual information from correlations.** Two variables can have $I(X;Y) > 0$ but zero linear correlation (think $Y = X^2$ with $X$ symmetric).

## 5.9 Chapter Summary

- Entropy: $H(X) = -\sum p(x) \log p(x)$ measures uncertainty.
- Cross-entropy: $H(p,q) = H(p) + D_{\mathrm{KL}}(p\|q)$.
- KL divergence is non-negative and zero iff distributions match.
- Mutual information: $I(X;Y) = H(X) + H(Y) - H(X,Y)$.
- Logs appear because they convert independence (multiplication) into addition.
- Master formula:

$$\boxed{D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \geq 0.}$$

## 5.10 Homework

**Problem 1.** **[Easy]** Compute the entropy of a uniform distribution over 8 outcomes (in bits).

**Problem 2.** **[Easy]** Compute $H(X)$ for $X \sim \mathrm{Bern}(0.25)$ in bits.

**Problem 3.** **[Easy]** A 4-class classifier predicts $\hat{y} = (0.4, 0.3, 0.2, 0.1)$. The true class is 2 (one-hot). Compute the cross-entropy loss in nats.

**Problem 4.** **[Medium]** For $p = (0.5, 0.5)$ and $q = (0.9, 0.1)$, compute both $D_{\mathrm{KL}}(p\|q)$ and $D_{\mathrm{KL}}(q\|p)$ in nats.

**Problem 5.** **[Medium]** Show that $H(X) \leq \log n$ where $n$ is the number of outcomes, with equality iff $X$ is uniform.

**Problem 6.** **[Medium, AI]** A language model assigns probability 0.001 to the correct next token. What is the cross-entropy loss in nats?

**Problem 7.** **[Medium, AI]** Two image features $X$ and $Y$ have $I(X;Y) = 0.5$ nats. Convert to bits.

**Problem 8.** **[Hard]** Prove that mutual information is symmetric: $I(X;Y) = I(Y;X)$.

**Problem 9.** **[Hard]** Compute the entropy of a Gaussian $\mathcal{N}(\mu, \sigma^2)$ in nats. Show it is $\frac{1}{2}\log(2\pi e\sigma^2)$.

**Problem 10.** **[Hard, AI]** A VAE has an encoder $q(z \mid x) = \mathcal{N}(\mu_\phi(x), \mathrm{diag}(\sigma_\phi^2(x)))$ and prior $p(z) = \mathcal{N}(0, \mathbf{I})$. Show that $D_{\mathrm{KL}}(q(z \mid x) \| p(z)) = \frac{1}{2}\sum_i(\mu_i^2 + \sigma_i^2 - 1 - \log\sigma_i^2)$.

# Chapter 6

# Discrete Mathematics

## 6.1 Opening Hook

When you scroll through a social network, every person is a node and every friendship an edge. When DeepMind's AlphaFold predicted protein structures, it ran message-passing on a graph of amino acids. When a search engine answers your query, it traverses an inverted index built from sets of documents. Discrete mathematics is the language of structure: of things that come in pieces and connect to each other. Without it, there are no graph neural networks, no symbolic reasoning, no efficient algorithms.

## 6.2 Why This Matters for AI

**Graphs are everywhere.** Social networks, knowledge bases, scene graphs, molecules, and computation graphs are all graphs. Graph neural networks operate on them directly.

**Sets and logic underpin reasoning.** Symbolic AI, theorem provers, and constraint solvers are pure discrete math.

**Combinatorics counts things.** How many ways can a permutation arise? How many parameters does a transformer have? How many valid moves in a game? Combinatorics answers these.

**Search algorithms drive planning.** A* on a graph is the heart of robotics planning, game AI, and route finding.

**Complexity tells you what is feasible.** Knowing whether a problem is $O(n)$ or $O(2^n)$ is the difference between a usable model and an unusable one.

## 6.3 Core Concepts

### 6.3.1 Sets

A *set* is an unordered collection of distinct elements: $A = \{1, 2, 3\}$. We write $x \in A$ to say $x$ is an element of $A$.

Standard operations: union $A \cup B$, intersection $A \cap B$, difference $A \setminus B$, complement $A^c$, Cartesian product $A \times B$.

The cardinality $|A|$ is the number of elements. The power set $2^A$ is the set of all subsets; $|2^A| = 2^{|A|}$.

### 6.3.2 Logic

Propositional logic builds compound statements from primitives using $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT), $\Rightarrow$ (implies), and $\Leftrightarrow$ (iff). A *tautology* is true under every assignment.

Quantifiers extend logic: $\forall x\, P(x)$ (for all), $\exists x\, P(x)$ (there exists). Negation flips quantifiers: $\neg\forall x P(x) \equiv \exists x\, \neg P(x)$.

### 6.3.3 Combinatorics

Three counting principles:

- **Sum rule.** If you can do $A$ in $m$ ways or $B$ in $n$ ways (mutually exclusive), there are $m + n$ ways total.
- **Product rule.** If you can do $A$ in $m$ ways and then $B$ in $n$ ways, there are $mn$ ways for the pair.
- **Permutations and combinations.**

$$P(n, k) = \frac{n!}{(n-k)!}, \qquad \binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

### 6.3.4 Graphs

**Definition 6.1** (Graph)**.** A *graph* $G = (V, E)$ is a set of vertices $V$ and a set of edges $E \subseteq V \times V$. The graph is *undirected* if edges are unordered pairs.

A graph can be encoded as an adjacency matrix $\mathbf{A}$ with $A_{ij} = 1$ if $(i, j) \in E$, else 0.

A *path* is a sequence of vertices connected by edges. A graph is *connected* if every pair of vertices is joined by a path.

The *degree* of a vertex is the number of edges incident to it.

### 6.3.5 Trees

A *tree* is a connected, acyclic graph. A tree on $n$ vertices has exactly $n - 1$ edges. Trees are the data structure underneath syntax parsers, decision trees, and Monte Carlo Tree Search.

### 6.3.6 Big-O complexity

We say $f(n) = O(g(n))$ if there exist $c, n_0 > 0$ such that $f(n) \leq c\, g(n)$ for all $n \geq n_0$. In words: $f$ grows no faster than $g$, ignoring constants.

Quick reference:

- $O(1)$: constant (hash lookup).
- $O(\log n)$: logarithmic (binary search).
- $O(n)$: linear (single pass).
- $O(n \log n)$: efficient sort.
- $O(n^2)$: nested loops, attention.
- $O(2^n)$: exponential, infeasible for large $n$.

## 6.4 Step-by-Step Derivations

### 6.4.1 Derivation 1: $|A \cup B| = |A| + |B| - |A \cap B|$

Partition $A \cup B$ into three disjoint pieces: $A \setminus B$, $B \setminus A$, and $A \cap B$. Then

$$|A \cup B| = |A \setminus B| + |B \setminus A| + |A \cap B| \qquad \text{(disjoint union)} \tag{6.1}$$

$$|A| = |A \setminus B| + |A \cap B| \qquad \text{(partition of } A\text{)} \tag{6.2}$$

$$|B| = |B \setminus A| + |A \cap B| \qquad \text{(partition of } B\text{)} \tag{6.3}$$

Add the second and third equations and subtract the first:

$$|A| + |B| - |A \cup B| = |A \cap B|, \quad \text{i.e.} \quad |A \cup B| = |A| + |B| - |A \cap B|.$$

### 6.4.2 Derivation 2: Number of subsets is $2^n$

A subset of an $n$-element set is determined by, for each element, choosing whether to include it. Two choices per element, $n$ elements, multiply by the product rule:

$$2 \cdot 2 \cdot \cdots \cdot 2 = 2^n.$$

### 6.4.3 Derivation 3: Tree on $n$ vertices has $n - 1$ edges

Induct on $n$.

Base: $n = 1$. A single vertex, zero edges. Check.

Step: assume any tree on $k$ vertices has $k - 1$ edges. Take a tree $T$ on $k + 1$ vertices. Every tree has at least one *leaf* (a vertex of degree 1); call it $v$. Remove $v$ and its single edge. The remaining graph is still connected (only $v$'s edge was removed) and acyclic (removing edges cannot create cycles), so it is a tree on $k$ vertices with, by induction, $k - 1$ edges. Adding $v$ back restores one edge, giving $(k - 1) + 1 = k = (k + 1) - 1$ edges in $T$.

### 6.4.4 Derivation 4: The Handshake Lemma

In any graph, the sum of all vertex degrees equals twice the number of edges:

$$\sum_{v \in V} \deg(v) = 2|E|.$$

The proof is a counting argument so direct it almost feels like cheating. Imagine walking through the graph and, for each edge $e = \{u, w\}$, putting one tally mark on $u$ and one tally mark on $w$. After processing every edge, each vertex $v$ has accumulated exactly $\deg(v)$ tally marks (one per edge incident to it). So

$$\text{total tally marks counted by vertex} = \sum_{v} \deg(v), \tag{6.4}$$

$$\text{total tally marks counted by edge} = 2|E| \qquad \text{(2 marks per edge).} \tag{6.5}$$

Both sums count the same pile of marks, so they must be equal.

**Immediate corollary (used everywhere in graph theory):** the number of vertices with odd degree must be even. Why? The sum on the left is even (it equals $2|E|$). Even-degree vertices contribute even numbers; odd-degree vertices contribute odd numbers. A sum of integers is even only if an even count of them are odd. So there cannot be a graph with exactly three people who have shaken an odd number of hands.

This lemma underlies analysis of network traffic, scheduling problems, and the very definition of an Eulerian circuit (which exists iff every vertex has even degree).

### 6.4.5 Derivation 5: Why Binary Search is $O(\log n)$

Algorithm complexity is where calculus, combinatorics, and discrete recurrences meet. Here we derive the running time of binary search from first principles.

Binary search looks for a target in a sorted array of length $n$. Each step compares the target to the middle element. If they match, done. Otherwise the target lies in either the left or right half, and we recurse on that half. Let $T(n)$ denote the worst-case number of comparisons. Then

$$T(n) = T(n/2) + 1, \qquad \text{recursive step (one comparison + half-size subproblem)} \tag{6.6}$$

$$T(1) = 1. \qquad \text{base case} \tag{6.7}$$

Solve by repeated substitution. Apply the recurrence $k$ times:

$$\begin{aligned} T(n) &= T(n/2) + 1 && && (6.8)\\ &= T(n/4) + 2 && \text{substitute } T(n/2) = T(n/4) + 1 && (6.9)\\ &= T(n/8) + 3 && && (6.10)\\ &\vdots && && (6.11)\\ &= T(n/2^k) + k. && && (6.12) \end{aligned}$$

The recursion stops when the subarray has size 1, i.e. when $n/2^k = 1$, which means $k = \log_2 n$. Therefore

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n = O(\log n).$$

This is why a search through one billion records takes only about 30 comparisons ($\log_2 10^9 \approx 29.9$). Every halving algorithm in machine learning — nearest-neighbor lookup in a kd-tree, decision trees, hierarchical softmax used in word embeddings — inherits this logarithmic scaling. It is the single most important reason why "divide and conquer" is a productive design strategy.

**Master theorem note.** The recurrence $T(n) = a\,T(n/b) + f(n)$ generalizes this argument: with $a$ subproblems each of size $n/b$ and $f(n)$ work per merge, the total work is determined by comparing $f(n)$ to $n^{\log_b a}$. For binary search ($a = 1, b = 2, f(n) = 1$): $n^{\log_2 1} = 1$, matching $f(n)$, giving $\Theta(\log n)$. For merge sort ($a = 2, b = 2, f(n) = n$): $n^{\log_2 2} = n$, also matching, giving $\Theta(n \log n)$.

## 6.5 Worked Examples

**Example 6.1: Inclusion-exclusion**

A class of 30 students. 20 play chess, 15 play cricket, 8 play both. How many play at least one?
**Solution.**

$$|C \cup K| = 20 + 15 - 8 = 27.$$

**Example 6.2: Counting permutations**

How many ways can 5 people sit in a row?
**Solution.** $5! = 120$.

**Example 6.3: Counting combinations**

A class of 20 students; choose a committee of 3. How many possible committees?
**Solution.**

$$\binom{20}{3} = \frac{20!}{3! \cdot 17!} = \frac{20 \cdot 19 \cdot 18}{6} = 1140.$$

**Example 6.4: Adjacency matrix**

Write the adjacency matrix of a triangle (3 vertices, all pairs connected).
**Solution.**

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}.$$

The diagonal is zero (no self-loops); the matrix is symmetric (undirected).

**Example 6.5: Big-O of nested loops**

What is the time complexity of the following pseudocode?

```
for i = 1 to n
    for j = 1 to n
        for k = 1 to log n
            do constant work
```

**Solution.** The outer loops contribute $n \cdot n = n^2$. The inner loop contributes $\log n$. Total: $O(n^2 \log n)$.

## 6.6 Real-World AI Applications

**Graph Neural Networks.** GNNs treat each node as a vector and update it by aggregating neighbours. The aggregation uses the graph's adjacency structure; the update uses a small neural net. Applications include molecule property prediction, traffic forecasting, and recommender systems.

**A* search.** Robotics path planners and game AI use A* on grid graphs. The algorithm relies on graph theory (vertices as states, edges as actions) and on a priority queue, an example of the data-structure side of discrete math.

**Decision trees and random forests.** A decision tree is a rooted tree where each internal node tests a feature and each leaf predicts a class. Random forests average many such trees. Tree depth and node counts are direct combinatorial quantities.

**Knowledge graphs.** Systems like Google Knowledge Graph and Wikidata represent facts as $(s, r, o)$ triples, equivalent to labeled directed graphs. Embedding entities into vector spaces (TransE, RotatE) is graph theory meeting linear algebra.

## 6.7 Visual Explanation

## 6.8 Common Pitfalls

**Pitfall 1: Confusing permutations with combinations.** Order matters in permutations; not in combinations. The number of poker hands is $\binom{52}{5}$, not $52!/47!$.

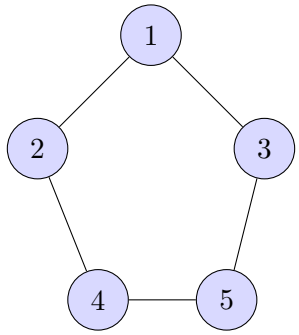



Figure 6.1: An undirected graph with five vertices and five edges.

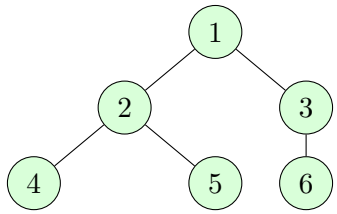



Figure 6.2: A rooted tree: 6 vertices, 5 edges, exactly one path between any pair.

**Pitfall 2: Off-by-one in trees.** A tree on $n$ vertices has $n-1$ edges. Forgetting the minus one is a classic source of bugs.

**Pitfall 3: Treating directed and undirected graphs alike.** The adjacency matrix of an undirected graph is symmetric; that of a directed graph need not be.

**Pitfall 4: Misreading Big-O.** $O(n^2)$ does not mean "the algorithm takes exactly $n^2$ steps". It is an upper bound up to a constant.

**Pitfall 5: Believing all graph problems are easy.** Many are NP-hard (graph coloring, traveling salesman). Approximate or heuristic methods are usually the practical answer.

**Pitfall 6: Forgetting empty set has $2^0 = 1$ subset.** The empty set has exactly one subset, itself. This matters when applying recursion.

## 6.9 Chapter Summary

- Sets: union, intersection, complement, power set with $2^n$ subsets.
- Logic: $\wedge, \vee, \neg, \Rightarrow$; $\forall, \exists$ with negation rules.
- Counting: $n!$, $P(n,k)$, $\binom{n}{k}$.
- Graphs $G = (V, E)$, adjacency matrix, paths, trees ($n$ vertices, $n-1$ edges).
- Big-O describes asymptotic growth; common classes from $O(1)$ to $O(2^n)$.
- Inclusion-exclusion:

$$\boxed{|A \cup B| = |A| + |B| - |A \cap B|.}$$

## 6.10 Homework

**Problem 1.** **[Easy]** List all subsets of $\{a, b, c\}$.

**Problem 2.** **[Easy]** Compute $\binom{6}{2}$.

**Problem 3.** **[Easy]** Negate the statement $\forall x\, P(x)$.

**Problem 4.** **[Medium]** A coffee shop offers 5 sizes and 8 syrups; you can pick at most one syrup. How many drink choices are there?

**Problem 5.** **[Medium]** Show by induction that $1 + 2 + \cdots + n = n(n + 1)/2$.

**Problem 6.** **[Medium, AI]** The attention mechanism in a transformer is $O(n^2)$ in the sequence length $n$. Explain in one sentence why.

**Problem 7.** **[Medium, AI]** A graph neural network performs a message-passing step in time $O(|V| + |E|)$. For a graph of 1 million nodes and 5 million edges, how many basic operations approximately?

**Problem 8.** **[Hard]** Show that any tree with at least two vertices has at least two leaves.

**Problem 9.** **[Hard]** Prove that for finite sets, $|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |B \cap C| - |A \cap C| + |A \cap B \cap C|$.

**Problem 10.** **[Hard, AI]** A knowledge graph has $|V| = 10^7$ entities and $|E| = 10^8$ triples. A naive algorithm checks every pair of entities: how many pair checks? Could you tractably run it on a single laptop today?

# Appendix A

# Math Refresher

## A.1 High-school algebra

A quick review of the rules you used in school.

**Exponents.** For real $a > 0$ and any reals $m, n$:

$$a^m \cdot a^n = a^{m+n}, \quad (a^m)^n = a^{mn}, \quad a^0 = 1, \quad a^{-n} = \frac{1}{a^n}.$$

**Logarithms.** The log is the inverse of the exponential: $\log_b(b^x) = x$. For natural log, base is $e$.

$$\begin{aligned} \log(xy) &= \log x + \log y, \\ \log(x/y) &= \log x - \log y, \\ \log(x^a) &= a \log x. \end{aligned}$$

**Quadratic formula.** The roots of $ax^2 + bx + c = 0$ are

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

**Polynomial expansion.**

$$\begin{aligned} (a+b)^2 &= a^2 + 2ab + b^2, \\ (a-b)^2 &= a^2 - 2ab + b^2, \\ (a+b)(a-b) &= a^2 - b^2, \\ (a+b)^3 &= a^3 + 3a^2b + 3ab^2 + b^3. \end{aligned}$$

**Summation.**

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}, \quad \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}, \quad \sum_{i=0}^{n-1} r^i = \frac{1-r^n}{1-r} \text{ for } r \neq 1.$$

## A.2 Trigonometry essentials

**Pythagorean identity.** $\sin^2\theta + \cos^2\theta = 1$.

**Angle addition.**

$$\sin(\alpha \pm \beta) = \sin\alpha\cos\beta \pm \cos\alpha\sin\beta,$$
$$\cos(\alpha \pm \beta) = \cos\alpha\cos\beta \mp \sin\alpha\sin\beta.$$

**Double angle.** $\sin 2\theta = 2\sin\theta\cos\theta$, $\cos 2\theta = \cos^2\theta - \sin^2\theta = 1 - 2\sin^2\theta$.

**Common values.**

| $\theta$ | $0$ | $\pi/6$ | $\pi/4$ | $\pi/3$ | $\pi/2$ |
|---|---|---|---|---|---|
| $\sin\theta$ | $0$ | $1/2$ | $\sqrt{2}/2$ | $\sqrt{3}/2$ | $1$ |
| $\cos\theta$ | $1$ | $\sqrt{3}/2$ | $\sqrt{2}/2$ | $1/2$ | $0$ |

# Appendix B

# Greek Alphabet Reference

The Greek letters that appear most often in AI mathematics.

| Lower | Upper | Name | Pronunciation |
|---|---|---|---|
| $\alpha$ | $A$ | Alpha | AL-fa |
| $\beta$ | $B$ | Beta | BAY-ta |
| $\gamma$ | $\Gamma$ | Gamma | GAM-a |
| $\delta$ | $\Delta$ | Delta | DEL-ta |
| $\epsilon, \varepsilon$ | $E$ | Epsilon | EP-si-lon |
| $\zeta$ | $Z$ | Zeta | ZAY-ta |
| $\eta$ | $H$ | Eta | AY-ta (often used for learning rate) |
| $\theta, \vartheta$ | $\Theta$ | Theta | THAY-ta (parameters) |
| $\iota$ | $I$ | Iota | eye-OH-ta |
| $\kappa$ | $K$ | Kappa | KAP-a |
| $\lambda$ | $\Lambda$ | Lambda | LAM-da (eigenvalue, regularizer) |
| $\mu$ | $M$ | Mu | MEW (mean) |
| $\nu$ | $N$ | Nu | NEW |
| $\xi$ | $\Xi$ | Xi | KSI |
| $\pi$ | $\Pi$ | Pi | PIE (also for policy) |
| $\rho$ | $P$ | Rho | ROW (correlation) |
| $\sigma$ | $\Sigma$ | Sigma | SIG-ma (std dev, sigmoid, sum) |
| $\tau$ | $T$ | Tau | TOW |
| $\upsilon$ | $\Upsilon$ | Upsilon | UP-si-lon |
| $\phi, \varphi$ | $\Phi$ | Phi | FIE (parameters) |
| $\chi$ | $X$ | Chi | KAI |
| $\psi$ | $\Psi$ | Psi | SIGH |
| $\omega$ | $\Omega$ | Omega | oh-MAY-ga (sample space) |

# Appendix C

# Further Reading

A curated list of resources to deepen each topic in this book.

1. **Strang, *Introduction to Linear Algebra.*** The classic friendly textbook on linear algebra; pair with Strang's free MIT OpenCourseWare video lectures.
2. **Axler, *Linear Algebra Done Right.*** A more abstract treatment, ideal once you are comfortable with computations and want the theory.
3. **Wasserman, *All of Statistics.*** A whirlwind tour of statistics for computer scientists; concise and complete.
4. **Bishop, *Pattern Recognition and Machine Learning.*** The standard graduate reference; first three chapters are particularly good for probability and Bayesian thinking.
5. **Boyd & Vandenberghe, *Convex Optimization.*** The definitive book on convex optimization; the free PDF and Stanford course are gold.
6. **Goodfellow, Bengio & Courville, *Deep Learning.*** Part I covers exactly the math in this handbook at a slightly higher level; freely available online.
7. **MacKay, *Information Theory, Inference, and Learning Algorithms.*** A delightful, opinionated tour through information theory and Bayesian inference.
8. **Cover & Thomas, *Elements of Information Theory.*** The standard reference for entropy, KL, mutual information, and channel capacity.
9. **Rosen, *Discrete Mathematics and Its Applications.*** A thorough first textbook on sets, logic, combinatorics, and graphs.
10. **Hamilton, *Graph Representation Learning.*** A short, modern survey of graph neural networks and their mathematical foundations; freely available.
11. **3Blue1Brown YouTube channel.** Visual intuition for linear algebra, calculus, and neural networks. Watch the "Essence of Linear Algebra" and "Essence of Calculus" series first.
12. **Andrew Ng's *Machine Learning Specialization* (Coursera).** A gentle hands-on introduction to ML that practices much of the math from this handbook.